

PDEBench 공개 Burgers 데이터 ROM 리뷰

이 문서는 기존 자체 생성 Burgers 데이터 결과를 제외하고, 공개 PDEBench 1D Burgers 데이터셋으로 수행한 ROM 결과만 정리한다.

사용한 공개 데이터는 PDEBench의 1D_Burgers_Sols_Nu0.01.hdf5이다. 원본 HDF5에서 동일한 subset을 추출한 뒤, 그 같은 processed HDF5 파일에 POD reconstruction, DMD-based prediction, Conv1D Autoencoder reconstruction 및 latent linear rollout을 적용했다.

핵심 결론은 다음과 같다.

- 세 방법 모두 같은 public subset과 같은 held-out case_index=14를 사용했다.
- POD는 시간 예측이 아니라 snapshot projection reconstruction baseline이다.
- DMD는 linear rollout prediction이므로 시간이 지나면서 prediction error가 증가한다.
- Conv1D AE는 reconstruction과 latent linear rollout에서 서로 다른 오차 양상을 보였다.

1. 공개 데이터셋 출처

PDEBench 원본 데이터

공개 데이터셋: PDEBench

원본 파일: 1D_Burgers_Sols_Nu0.01.hdf5

원본 URL: <https://darus.uni-stuttgart.de/api/access/datafile/281363>

원본 tensor shape: [10000, 201, 1024, 1]

프로젝트에서 사용한 subset

원본 저장 위치: data/external/pdebench/1D_Burgers_Sols_Nu0.01.hdf5

processed subset: data/processed/burgers/pdebench_burgers_nu0.01_subset.h5

subset shape: (64, 101, 128)

subset rule: {'case_offset': 0, 'n_cases_requested': 64, 'n_cases_used': 64, 'time_stride': 2, 'space_stride': 8, 'source_case_indices': [437, 636, 677, 854, 886, 918, 936, 1275, 1541, 1648, 1818, 1944, 2003, 2265, 2760, 3538, 3635, 3666, 3692, 4009, 4304, 4362, 4419, 4449, 4484, 4489, 4662, 4973, 4982, 5108, 5236, 5435, 5461, 5529, 6301, 6414, 6505, 6769, 6827, 6934, 6990, 7139, 7320, 7430, 7446, 7564, 7573, 7589, 7691, 7772, 7785, 7822, 8197, 8254, 8359, 8536, 8559, 8853, 8916, 9222, 9230, 9675, 9689, 9705], 'split_seed': 42, 'selection_strategy': 'random_without_replacement', 'selection_seed': 42}

참고

긴 URL과 파일 경로는 전체 폭을 쓰는 카드형 레이아웃으로 내려 써서 잘리지 않도록 처리했다.

2. 세 방법에 같은 데이터가 적용되었는가?

POD

config: configs/burgers/pdebench_pod.yaml
data path: data/processed/burgers/pdebench_burgers_nu0.01_subset.h5
case_index: 14
rank: 8
동일 데이터 적용: yes

DMD

config: configs/burgers/pdebench_dmd.yaml
data path: data/processed/burgers/pdebench_burgers_nu0.01_subset.h5
case_index: 14
rank: 8
동일 데이터 적용: yes

Conv1D AE

config: configs/burgers/pdebench_autoencoder.yaml
data path: data/processed/burgers/pdebench_burgers_nu0.01_subset.h5
case_index: 14
latent_dim: 8
동일 데이터 적용: yes

참고

세 방법은 같은 processed HDF5와 같은 평가 case를 쓴다. 단, 학습/평가 의미는 다르다. POD는 train split 전체로 basis를 만들고 case 14를 projection reconstruction한다. DMD와 AE는 case 14 내부 시간 구간에서 rollout 또는 latent dynamics를 평가한다.

3. PDEBench public subset split

split	case indices	개수
train	3, 4, 5, 7, 9, 15, 16 17, 18, 20, 21, 23, 24, 25 26, 27, 28, 29, 30, 31, 32 34, 37, 39, 40, 42, 44, 46 50, 51, 54, 55, 56, 58, 59 60, 61, 63	38
validation	0, 6, 10, 11, 19, 22, 35 38, 41, 47, 48, 52, 57	13
test	1, 2, 8, 12, 13, 14, 33 36, 43, 45, 49, 53, 62	13

64개 source trajectory는 seed 42로 비복원 무작위 추출했고 local split도 seed 42 permutation으로 만들었다. POD/DMD/AE config의 case_index=14는 test split에 포함된 held-out case이다.

4. 기본 대표 결과표: public-only case 14

method	rank/latent	reconstruction L2	rollout L2	final error	front MAE
POD	8	0.3096	N/A	0.1199	0.1067
DMD	8	0.1710	0.8868	1.2586	0.3973
Conv1D AE	8	0.2841	0.8807	1.0361	0.2673

이 표는 기본 파이프라인의 대표 결과다. 모두 같은 PDEBench processed dataset과 case 14를 사용하지만, POD의 final error는 final snapshot reconstruction error이고 DMD/AE의 final error는 rollout final error이다. 따라서 아래 controlled comparison 표와 분리해서 읽어야 한다.

5. 결과 해석

POD rank 8 reconstruction relative L2는 $3.096073e-01$ 이다. POD는 주어진 true snapshot을 POD basis에 projection해 복원하므로 안정적인 선형 baseline 역할을 한다.

DMD reconstruction relative L2는 $1.710189e-01$, rollout relative L2는 $8.867985e-01$, final rollout error는 $1.258604e+00$ 이다. 이는 linear time-evolution model에서 시간 구간에 따라 오차가 다르게 나타남을 보여준다.

Conv1D AE reconstruction relative L2는 $2.841150e-01$ 이다. 같은 latent_dim=8 조건에서 nonlinear decoder가 snapshot reconstruction을 어떻게 표현하는지 관찰할 수 있다.

AE latent linear rollout relative L2는 $8.806610e-01$, final rollout error는 $1.036132e+00$ 이다. 즉 reconstruction latent와 dynamics coordinate를 분리해서 해석해야 한다.

따라서 같은 공개 데이터셋에서 비교할 때, POD는 linear reconstruction baseline, DMD는 linear rollout baseline, Conv1D AE는 nonlinear reconstruction과 latent rollout 관찰 대상으로 구분해서 읽는다.

5A. 13개 test trajectory 전체 temporal extrapolation

DMD

rollout mean +/- std: 0.2335 +/- 0.2641
final mean +/- std: 0.2945 +/- 0.3867
front MAE median [IQR]: 0.2967 [0.2041, 0.3923]
smooth rollout mean: 0.0784
shock rollout mean: 0.3664
diverged cases: 0 / 13

AE latent rollout

rollout mean +/- std: 0.3112 +/- 0.3545
final mean +/- std: 0.3442 +/- 0.4085
front MAE median [IQR]: 0.2797 [0.2298, 0.3847]
smooth rollout mean: 0.1076
shock rollout mean: 0.4857
diverged cases: 0 / 13

참고

범위: All 13 test trajectories; case-specific temporal fit on indices 0:59 and autonomous evaluation on 60:100. 발산 정의: Any non-finite prediction or per-snapshot future relative L2 greater than 10. 대표 extreme case 하나의 결과를 전체 성능으로 일반화하지 않기 위해 모든 test trajectory를 집계했다.

6. comparison CSV에 남은 public-only rows

pdebench_burgers_ae_latent8

method: autoencoder
reconstruction L2: 0.2841
rollout L2: 0.8807
final error: 1.0361

pdebench_burgers_dmd_rank8

method: dmd
reconstruction L2: 0.1710
rollout L2: 0.8868
final error: 1.2586

pdebench_burgers_pod_rank8

method: pod
reconstruction L2: 0.3096
rollout L2: N/A
final error: 0.1199

7. 코드 로직 검토: 어떤 파일이 무엇을 하는가

공개 데이터 준비

코드 경로: `scripts/prepare_pdebench_burgers.py`

설정 경로: `configs/burgers/pdebench_prepare.yaml`

로직 판단: PDEBench 원본 HDF5에서 지정한 subset을 읽어 프로젝트 표준 HDF5 형식으로 저장하므로 재실행성과 비교 기준이 일관적이다.

확인 결과: $u=(64, 101, 128)$, train=38 cases, val=13 cases, test=13 cases로 저장되어 세 방법이 같은 processed file을 사용한다.

POD reconstruction

코드 경로: `scripts/train_pod.py`, `src/rom_bench/models/pod.py`

설정 경로: `configs/burgers/pdebench_pod.yaml`

로직 판단: train split snapshot으로 SVD basis를 만들고 held-out case 14를 rank 8 basis에 projection한다.

구현 방향은 POD reconstruction 목적에 맞다.

중요한 해석: POD 결과는 시간 예측이 아니라 true snapshot을 알고 있을 때의 projection reconstruction이다. 따라서 rollout error가 아니라 reconstruction baseline으로 해석해야 한다.

DMD-based prediction

코드 경로: `scripts/train_dmd.py`, `src/rom_bench/models/dmd.py`

설정 경로: `configs/burgers/pdebench_dmd.yaml`

로직 판단: case 14의 앞쪽 시간 구간으로 rank 8 exact DMD를 학습한 뒤 이후 시간을 free rollout한다. DMD prediction 실험 목적에 맞다.

중요한 해석: Burgers 방정식은 nonlinear이지만 DMD는 하나의 linear time-advance operator로 근사한다. 그래서 학습 구간 재구성과 장시간 rollout에서 서로 다른 오차 양상이 나타날 수 있다.

7. 코드 로직 검토: 어떤 파일이 무엇을 하는가 계속

Conv1D Autoencoder

코드 경로: scripts/train_autoencoder.py, src/rom_bench/models/autoencoder_1d.py,
src/rom_bench/models/latent_dynamics.py

설정 경로: configs/burgers/pdebench_autoencoder.yaml

로직 판단: PyTorch Conv1D encoder-decoder로 snapshot reconstruction을 학습하고, latent vector에는 별도 linear dynamics를 맞춰 rollout한다. reconstruction 실험 로직은 맞다.

중요한 해석: AE는 nonlinear decoder로 field reconstruction을 수행하지만, latent 좌표가 자동으로 선형 시간좌표가 되는 것은 아니다. latent linear rollout은 별도로 해석해야 한다.

공통 비교

코드 경로: scripts/compare_models.py

입력: metrics/pdebench_burgers_*_metrics.json

출력: metrics/burgers_model_comparison.csv

로직 판단: 세 방법의 metrics를 한 CSV로 모으는 로직은 맞다. 다만 POD reconstruction, DMD rollout, AE reconstruction/latent rollout은 물리적으로 같은 난이도의 문제는 아니므로 해석에서 구분해야 한다.

참고

종합 판단: 코드 흐름은 공개 데이터 준비, 동일 case 선택, 방법별 학습/평가, metrics 저장 순서로 일관적이다. 주의할 점은 숫자 하나로 방법의 우열을 정하는 것이 아니라 reconstruction과 rollout을 분리해서 읽어야 한다는 점이다.

8. 상세 코드 로직: 데이터 흐름과 평가 대상

전체 파이프라인의 출발점은 processed HDF5 파일이다. 이 파일에는 x , t , u , $\text{split/train_indices}$, split/val_indices , $\text{split/test_indices}$ 가 들어 있다. 여기서 u 의 shape은 [case 개수, 시간 snapshot 개수, 공간 격자점 개수] = [64, 101, 128]이다.

즉 한 개의 Burgers 해는 $u_{\text{case}} = u[\text{case_index}]$ 형태로 꺼내며, 이번 리뷰에서 세 방법은 모두 $\text{case_index}=14$ 를 사용한다. u_{case} 의 shape은 [101, 128]이므로, 시간 101개와 공간 격자점 128개를 가진 하나의 space-time field라고 보면 된다.

POD는 train split에 있는 여러 case의 snapshot을 모아서 basis를 만든다. 그 다음 test split에 포함된 case 14를

그 basis 위에 projection한다. 따라서 POD는 '처음 보는 test case를 train basis가 얼마나 잘 복원하는가'를 보는 구조다.

DMD는 case 14의 앞쪽 시간 구간을 학습 구간으로 사용한다. 앞쪽 snapshot pair로 선형 시간 전진 operator를 추정하고, 이후 시간은 true 값을 넣지 않고 free rollout한다. 따라서 DMD의 핵심 평가는 '시간이 지날수록 예측 오차가 어떻게 누적되는가'이다.

Autoencoder는 case 14의 snapshot을 Conv1D encoder-decoder로 압축하고 복원한다. encoder는 $u(x)$ 를 latent vector z 로 바꾸고, decoder는 z 에서 다시 $u(x)$ 를 만든다. 추가 rollout은 latent z 에 대해 linear dynamics를 맞춰서 수행한다.

평가 지표는 세 방법이 같은 true field와 비교되도록 저장된다. relative L2 error는 $\| \text{prediction} - \text{truth} \| / (\| \text{truth} \| + \text{epsilon})$ 으로 계산된다. front 위치 오차는 공간 기울기 $|du/dx|$ 가 큰 위치를 front로 보고, true front와 predicted front의 차이를 계산한다.

따라서 코드 로직 자체는 같은 공개 데이터, 같은 case, 같은 rank 또는 $\text{latent_dim}=8$ 을 기준으로 비교하도록 정리되어 있다. 다만 POD reconstruction, DMD rollout, AE latent rollout은 문제의 성격이 다르므로 결과 해석에서 반드시 구분해야 한다.

9. 상세 코드 로직: POD reconstruction

POD는 먼저 train snapshot들을 하나의 큰 행렬 X 로 만든다. 실제 코드에서는 각 snapshot $u(x)$ 가 길이 $n_x=128$ 인 벡터이고, 여러 시간과 여러 case의 snapshot을 행 방향으로 쌓는다. 따라서 X 의 shape은 $[n_samples, n_features]$ 이다.

mean subtraction이 켜져 있으면 train snapshot의 평균 field를 먼저 뺀다. 이렇게 하면 POD mode는 평균값 자체가 아니라 평균에서 벗어나는 주요 변형 패턴을 학습한다.

그 다음 SVD를 수행한다. 코드 기준으로는 $centered = U S V^h$ 이다. snapshot이 행이고 공간 격자가 열이므로, spatial mode는 U 가 아니라 V^h 의 행이다. 실제 구현도 $modes = V^h[:rank]$ 로 저장한다.

$rank=8$ 실험에서는 V^h 의 앞 8개 행을 Φ 로 사용한다. test snapshot u_true 가 들어오면 $u_centered = u_true - mean$ 을 만들고, 계수 $a = u_centered \Phi^T$ 를 계산한다. reconstruction은 $u_pred = a \Phi + mean$ 이다.

이 로직은 POD reconstruction으로 맞다. 왜냐하면 POD는 주어진 rank에서 train 데이터의 평균 제곱 projection error를 최소화하는 선형 basis를 찾는 방법이기 때문이다.

하지만 여기에는 시간 동역학 모델이 없다. POD reconstruction error가 어떤 값을 갖더라도, 그것만으로 미래를 예측했다는 뜻은 아니다.

현재 POD 결과는 '이미 주어진 snapshot을 rank 8 선형 공간에 어떻게 압축하고 복원했는가'를 의미한다.

moving front에서는 같은 모양이 오른쪽이나 왼쪽으로 조금 이동하는 현상이 생긴다. 선형 basis는 이동 자체를 자연스럽게 표현하지 못해서 여러 mode를 필요로 한다. 그래서 주어진 rank에서 전체 L2 error와 gradient/front 주변 error가 함께 나타날 수 있다.

10. 상세 코드 로직: DMD prediction

DMD는 시간 순서가 있는 snapshot pair를 사용한다. 학습 구간의 snapshot을 $X1 = [u_0, u_1, \dots, u_{m-1}]$, $X2 = [u_1, u_2, \dots, u_m]$ 처럼 두 행렬로 나눈다.

목표는 $X2$ 가 $A X1$ 과 비슷해지도록 선형 operator A 를 찾는 것이다. full A 를 직접 만들면 너무 클 수 있으므로, 코드에서는 SVD로 $\text{rank}=8$ truncation을 하고 reduced operator를 계산한다.

그 다음 reduced operator의 eigenvalue와 DMD mode를 구한다. eigenvalue의 크기는 mode가 시간에 따라 감쇠하는지 커지는지를 보여주고, angle은 진동 주파수와 관련된다.

reconstruction에서는 초기 amplitude를 계산한 뒤 DMD mode와 eigenvalue를 이용해 학습 구간 또는 전체 시간의 field를 만든다. rollout에서는 학습 이후 시점에 true snapshot을 다시 넣지 않고 eigenvalue를 계속 거듭제곱해서 미래를 생성한다.

이 로직은 DMD-based prediction으로 맞다. DMD는 nonlinear Burgers 방정식을 직접 풀지 않고, 관측된 snapshot 사이의 시간 전진을 선형 근사로 표현하는 방법이다.

다만 Burgers 방정식의 실제 변화에는 $u \frac{du}{dx}$ 라는 nonlinear 항과 viscosity diffusion이 함께 작용한다. 하나의 고정된 선형 operator로 이 효과를 모든 시간에 정확히 표현하기 어렵다.

그래서 DMD reconstruction error와 rollout error는 서로 다른 크기와 시간 분포를 가질 수 있다. 미래로 갈수록 front 위치, phase, amplitude, gradient가 조금씩 어긋나며 오차가 누적될 수 있기 때문이다.

11. 상세 코드 로직: Conv1D Autoencoder

Autoencoder는 각 1D snapshot $u(x)$ 를 입력으로 받는다. Conv1D encoder는 공간 방향의 국소 패턴, 예를 들면 완만한 영역과 급격한 front 주변 패턴을 convolution filter로 읽어 latent vector z 로 압축한다.

latent_dim=8이면 하나의 snapshot은 z_1 부터 z_8 까지 총 8개의 숫자로 표현된다. 이 숫자들은 사람이 미리 정한 물리량이 아니라, reconstruction loss를 줄이기 위해 neural network가 스스로 만든 내부 좌표다.

decoder는 이 8개 latent 숫자에서 다시 길이 128의 $u(x)$ 를 만든다. loss는 기본적으로 true field와 reconstructed field 사이의 MSE 또는 설정된 추가 loss를 줄이는 방향으로 계산된다.

학습 중에는 validation loss가 최소로 기록된 checkpoint를 best checkpoint라는 이름으로 저장한다. 이번 결과에서 backend가 pytorch_conv1d로 기록되어 있으므로, 실제 PyTorch Conv1D Autoencoder가 사용된 것이 맞다.

latent trajectory figure는 시간에 따라 $z_1, z_2, \dots, z_8$ 이 어떻게 변하는지 그린 것이다. 이 그래프는 AE가 field의 시간 변화를 latent 공간에서 어떤 경로로 표현하는지 보는 용도다.

중요한 점은 AE의 기본 목적이 reconstruction이라는 것이다. 즉 z 가 field를 잘 복원하도록 학습되었지, $z(t+1)=B z(t)$ 같은 간단한 선형 동역학을 따르도록 강하게 학습된 것은 아니다.

따라서 AE reconstruction error와 POD reconstruction error는 서로 다른 표현 방식의 차이를 보여준다.

nonlinear decoder는 rank 8 선형 basis와 다른 방식으로 field를 표현한다. 또한 latent linear rollout

error는 reconstruction error와 별개로 해석해야 한다. latent 공간의 압축 좌표가 곧 시간 예측 좌표는 아니기 때문이다.

12. 상세 코드 로직: 비교 결과를 읽는 기준

현재 comparison CSV는 method, rank 또는 latent_dim, reconstruction_relative_l2, rollout_relative_l2, final_rollout_error, front_position_mae 등을 한 표에 모은다. 이 정리는 실험 결과를 나란히 보기 위한 것이며, 모든 열을 같은 의미로 읽으면 안 된다.

POD의 reconstruction_relative_l2는 projection reconstruction error다. POD에는 별도 시간 전진 모델이 없으므로 rollout_relative_l2는 NaN 또는 N/A로 보는 것이 맞다.

DMD의 reconstruction_relative_l2는 modal reconstruction이 학습 데이터 구조를 얼마나 잘 맞췄는지 보여준다.

rollout_relative_l2와 final_rollout_error는 학습 이후 free prediction이 얼마나 무너지는지 보여준다.

AE의 reconstruction_relative_l2는 Conv1D encoder-decoder의 복원 성능이고, rollout_relative_l2는

latent linear dynamics까지 붙였을 때의 예측 성능이다. 이 둘은 같은 모델에서 나온 값이지만 의미는 다르다.

front_position_mae는 전체 L2 error와 다른 정보를 준다. 예를 들어 front 위치는 맞지만 amplitude가 틀리면 front error는 작고 L2 error는 클 수 있다. 반대로 전체 L2 error가 작아도 sharp gradient 위치가 조금 어긋나면 front error가 커질 수 있다.

따라서 현재 결과의 판단 기준은 다음과 같다. reconstruction 능력은 POD와 AE 중심으로 비교하고, 장시간 예측 안정성은 DMD rollout과 AE latent rollout 중심으로 비교한다. front 관련 평가는 전체 L2 error와 따로 읽는다.

13. 추가 검증 체크리스트

POD snapshot matrix 방향과 spatial mode

확인 결과: snapshots are rows: [n_samples, n_features]

spatial mode: `np.linalg.svd(centered)` returns U, S, Vh; code stores `modes=Vh[:rank]`, so

spatial modes are rows of Vh, not columns of U

판단: 기존 설명에서 U를 spatial mode처럼 쓴 부분은 일반적인 열-snapshot 관례 설명이었고, 실제 코드 기준으로는 Vh의 행이 spatial mode이다. PDF 설명을 이 기준으로 수정했다.

AE 학습 데이터와 case 14 포함 여부

case 14 split: test

HDF5 train split만 사용?: False

AE training snapshots: case 14, time indices 0..59

case 14 snapshot 포함?: yes, time indices 0..59 are used for AE training

판단: AE는 HDF5의 train case split만으로 학습된 것이 아니라, 평가 case 14의 앞 60개 시간 snapshot으로 학습되었다. 따라서 AE 결과는 temporal extrapolation 실험이지 case-generalization 실험은 아니다.

latent dynamics와 DMD/AE temporal split

latent dynamics fit: latent time indices 0..59

future snapshot fitting 사용?: False

DMD training snapshots: case 14, time indices 0..59

DMD와 AE temporal split 동일?: True

latent operator spectral radius: 0.9864

판단: 미래 test snapshot은 latent linear operator fitting에는 쓰이지 않았다. 다만 AE encoder-decoder 자체는 case 14의 앞쪽 시간구간을 보고 학습했다.

13. 추가 검증 체크리스트 계속

front detector 연속성

front method: max_gradient

dx at nx=128: 7.812e-03

max detector-position jump: 0.9922

large jump threshold: 0.0312

large jump count: 2

same front likely?: False

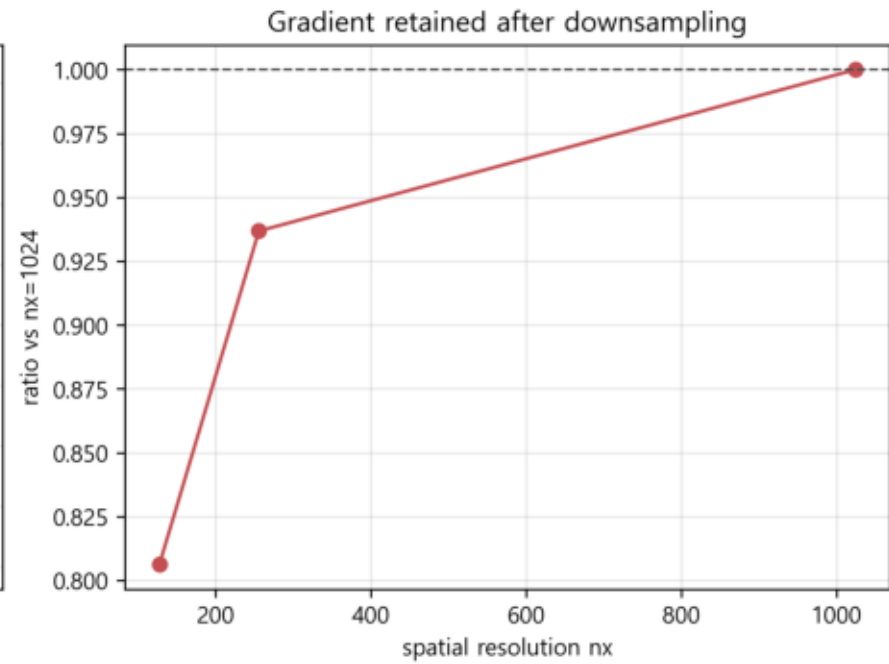
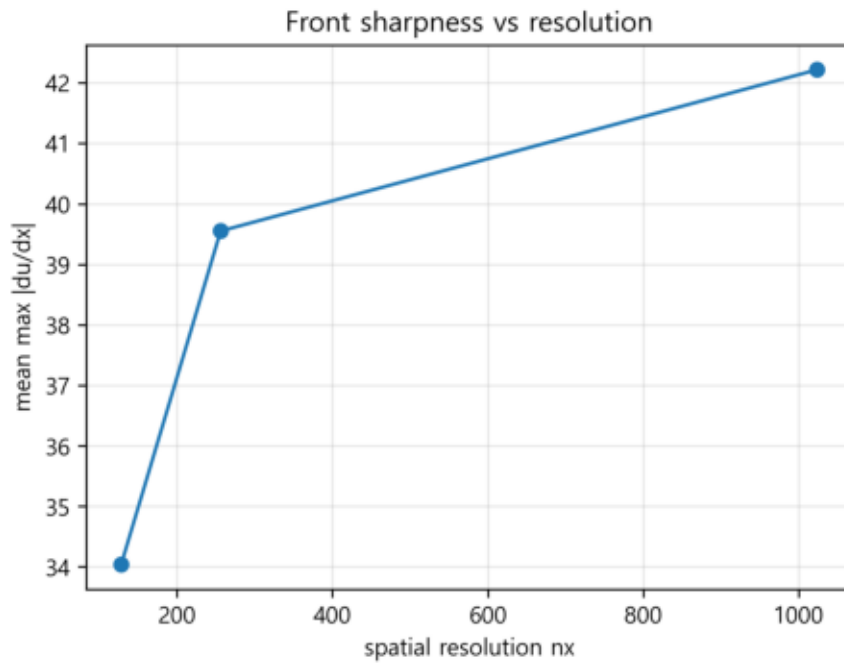
판단: max-gradient detector는 각 시점의 가장 큰 $|du/dx|$ 위치를 독립적으로 고른다. 여러 sharp region이 경쟁하거나 주기 경계를 넘으면 다른 위치로 전환될 수 있다. 따라서 이 jump는 물리적 front 속도나 해상도 오차가 아니다.

14. 공간해상도 128 / 256 / 1024 front gradient 비교

nx	stride	mean max du/dx	ratio vs 1024	max detector jump	jump count
128	8	34.0401	0.8063	0.9922	2
256	4	39.5513	0.9368	0.9961	2
1024	1	42.2185	1.0000	0.9893	82

원본 PDEBench source trajectory 2760를 같은 시간 stride로 읽고, 공간 stride만 8, 4, 1로 바꿔 nx=128, 256, 1024를 비교했다. ratio vs 1024가 1보다 작으면 downsampling 때문에 sharp front gradient가 약해졌다는 뜻이다. 마지막 두 열은 maximum-gradient 검출기의 위치 전환 진단이며 물리적 front 이동량으로 해석하지 않는다.

공간해상도에 따른 front sharpness 비교



$n_x=128$ 은 현재 ROM 실험에 쓰인 processed 해상도이고, $n_x=1024$ 는 PDEBench 원본 공간해상도이다. 이 그림은 downsampling이 $\max |du/dx|$ 를 얼마나 줄이는지 보여준다.

15. smooth/shock 및 dimension sweep 비교

이 비교는 어떤 방법의 순위를 매기기 위한 것이 아니다. 목적은 같은 공개 PDEBench 데이터 안에서 smooth-like case와 shock-like case를 명시적으로 나누고, front 주변에서 각 reconstruction 또는 rollout이 어떤 오차 모양을 만드는지 관찰하는 것이다.

사용한 데이터셋은 앞의 기본 실험과 동일하다. 같은 processed HDF5 안에서 test split의 mean max $|du/dx|$ 가 작은 case를 smooth-like, 큰 case를 shock-like로 골라 두 regime을 나란히 비교했다.

또한 POD와 AE reconstruction 비교가 공정하지 않다는 지적을 반영해, controlled comparison을 따로 만들었다. 이 controlled comparison에서는 POD와 AE 모두 같은 case의 같은 temporal training snapshots, 즉 time index 0..59만 사용한다.

따라서 이것은 다른 데이터셋을 쓴 별도 결과가 아니라, 같은 PDEBench 데이터셋 안에서 smooth/shock 및 dimension sweep 관찰 항목을 더 자세히 펼친 것이다.

16. smooth-like / shock-like case 선택

regime	case	mean max $ du/dx $	max $ du/dx $	max-gradient time index
smooth_like	45	0.7945	6.0907	1
shock_like	14	34.0401	131.4151	4

선택 기준은 test split 내부의 mean max $|du/dx|$ 이다. 값이 작은 case 45는 smooth-like, 값이 큰 case 14는 shock-like로 표시했다. 이 구분은 물리 regime label이 아니라, 현재 subset에서 gradient 강도 차이를 보기 위한 operational label이다.

17. controlled comparison 결과표: 동일 temporal split, dim=8

regime	method	post-split L2	final L2	front MAE	front-window L2
smooth_like	POD	1.396e-04	9.744e-05	N/A	3.360e-04
smooth_like	DMD	2.108e-04	1.312e-04	N/A	3.980e-04
smooth_like	AE	3.510e-04	3.463e-04	N/A	3.849e-04
shock_like	POD	0.5636	0.6005	0.1734	0.4301
shock_like	DMD	0.8868	1.2586	0.3973	0.4370
shock_like	AE	0.5458	0.6896	0.1547	0.2060

이 표는 기본 대표 결과표와 다른 controlled comparison이다. POD와 AE는 같은 case의 같은 time index 0..59로 reconstruction 모델을 맞췄고, DMD는 같은 구간으로 rollout operator를 맞췄다. smooth-like case는 뚜렷한 shock/front가 아니므로 front MAE를 N/A로 비활성화했다.

18. smooth/shock 및 dimension sweep에서 보이는 오차 양상

smooth-like case 45에서는 true field 자체의 gradient가 약하고 시간 변화도 완만하다. 그래서 front 주변 국부 오차 그림에서도 sharp shock failure보다는 작은 amplitude 차이나 완만한 profile 차이가 주로 보인다.

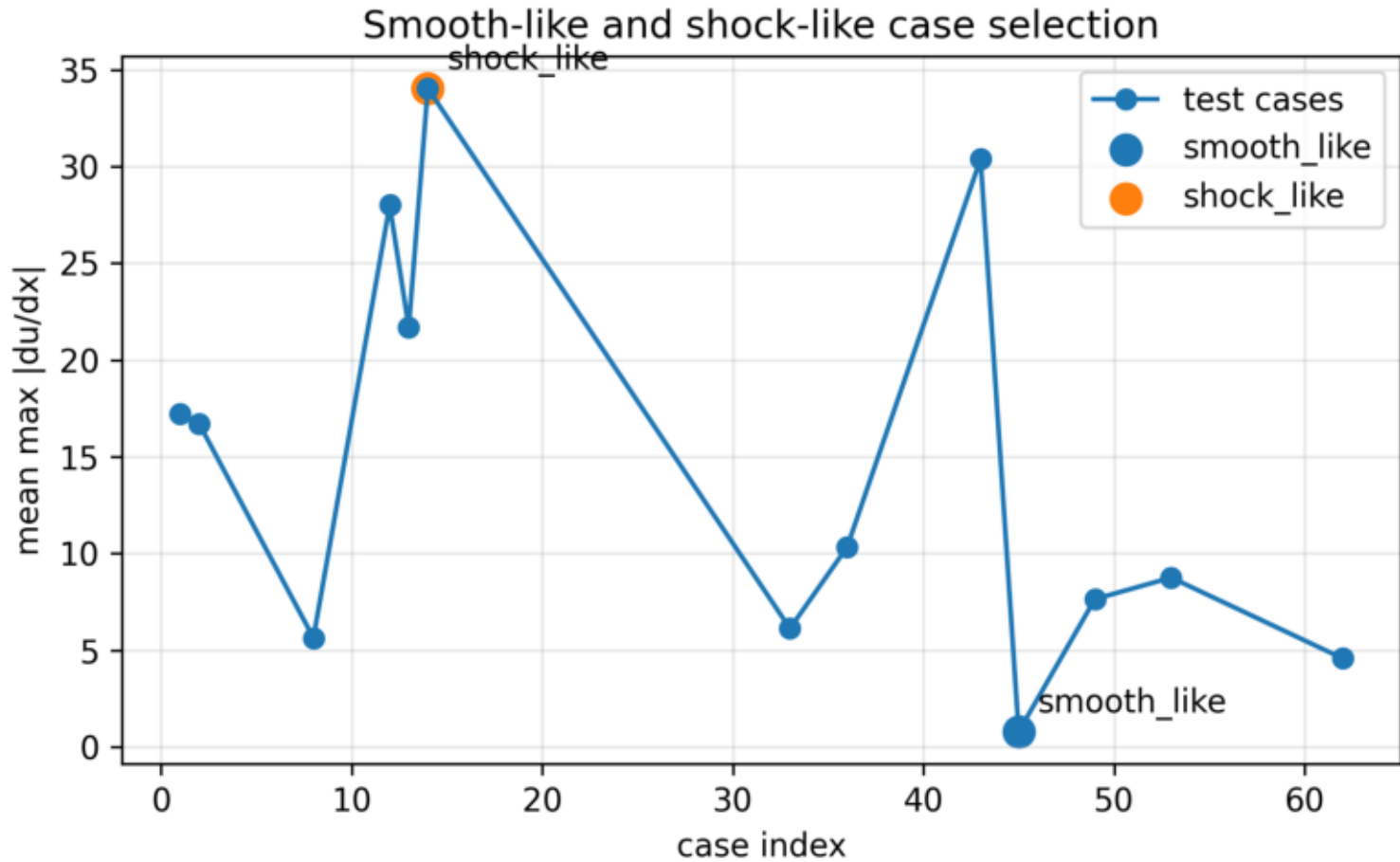
shock-like case 14에서는 mean max $|du/dx|$ 가 훨씬 크고, front 주변 국부 오차가 해석의 중심이 된다. 전체 L2 error만 보면 front가 어디서 어떻게 흐려졌는지 알기 어렵기 때문에, front overlay와 local absolute error를 함께 보도록 했다.

rank sweep과 latent-dimension sweep은 차원을 늘렸을 때 숫자가 어떻게 변하는지뿐 아니라, shock-like case에서 front-window error와 gradient error가 어떻게 남는지 보기 위한 것이다.

동일 학습 조건 비교에서는 POD와 AE 모두 같은 case-specific temporal training snapshots만 사용한다. 따라서 기존 기본 파이프라인의 POD/AE 비교보다 reconstruction 조건이 더 직접적으로 맞춰져 있다.

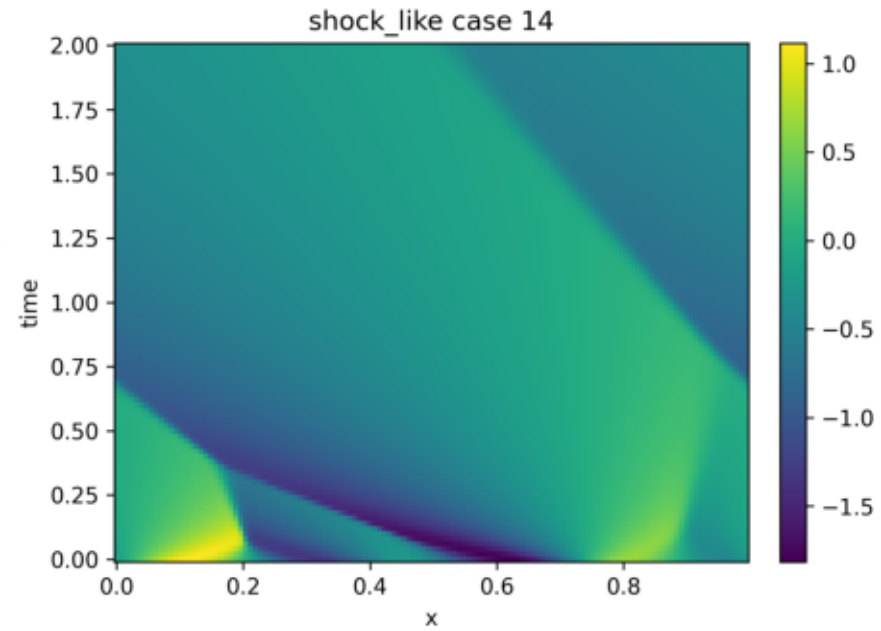
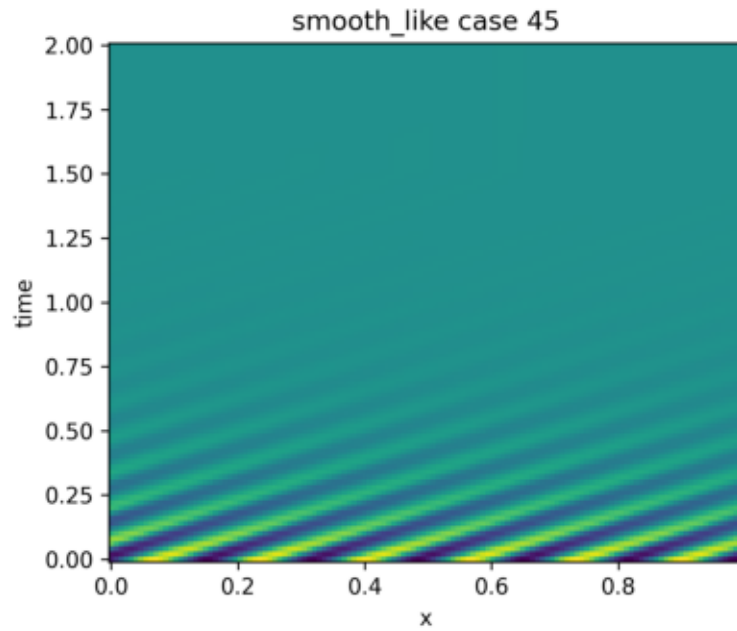
이 결과는 방법의 우열 판단이 아니라, smooth-like field와 shock-like field에서 오차가 나타나는 위치와 형태를 비교하기 위한 자료다.

smooth-like / shock-like case 선택 기준



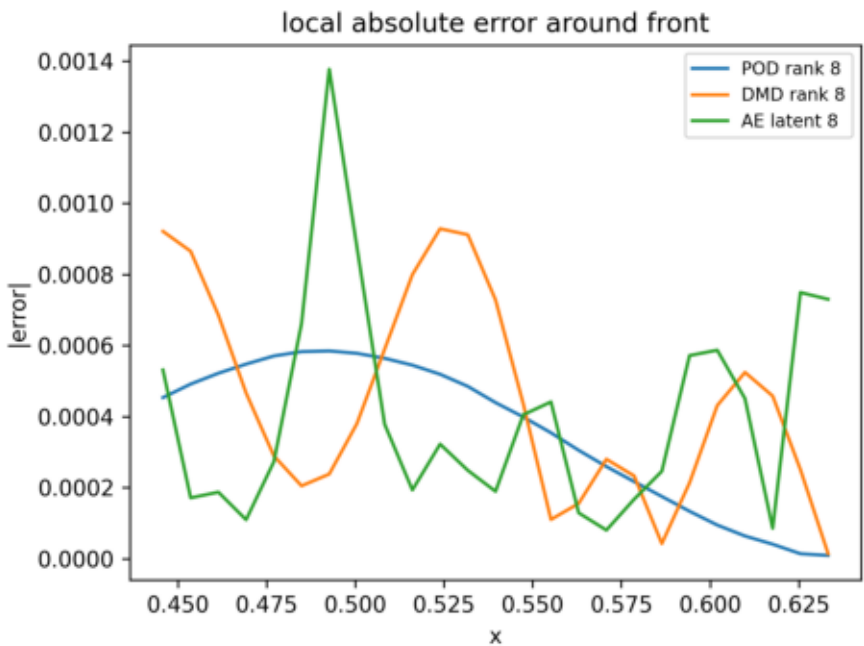
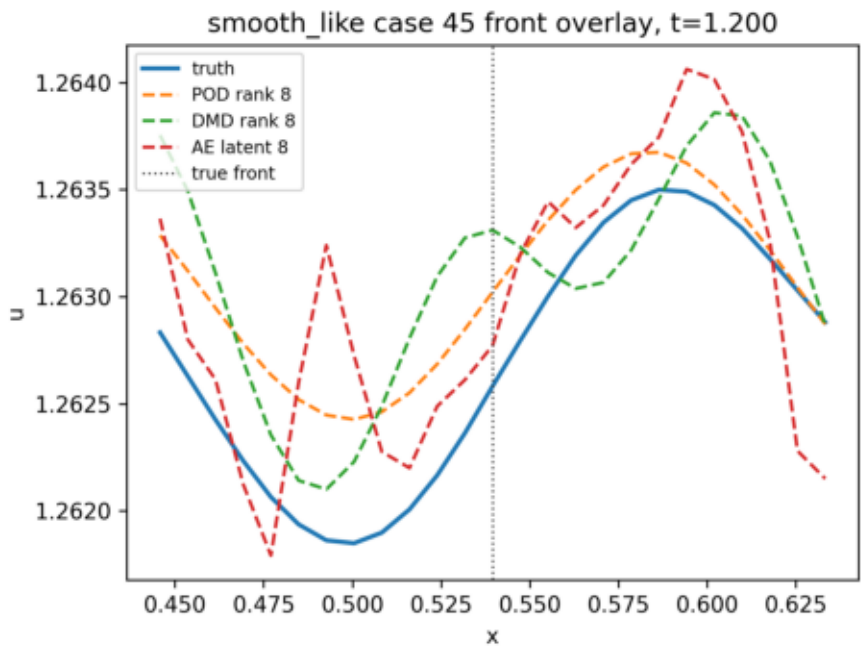
test split case들의 mean max $|du/dx|$ 를 표시하고, smooth/shock 비교에 사용한 smooth-like case 45와 shock-like case 14를 강조했다.

smooth-like와 shock-like true space-time field



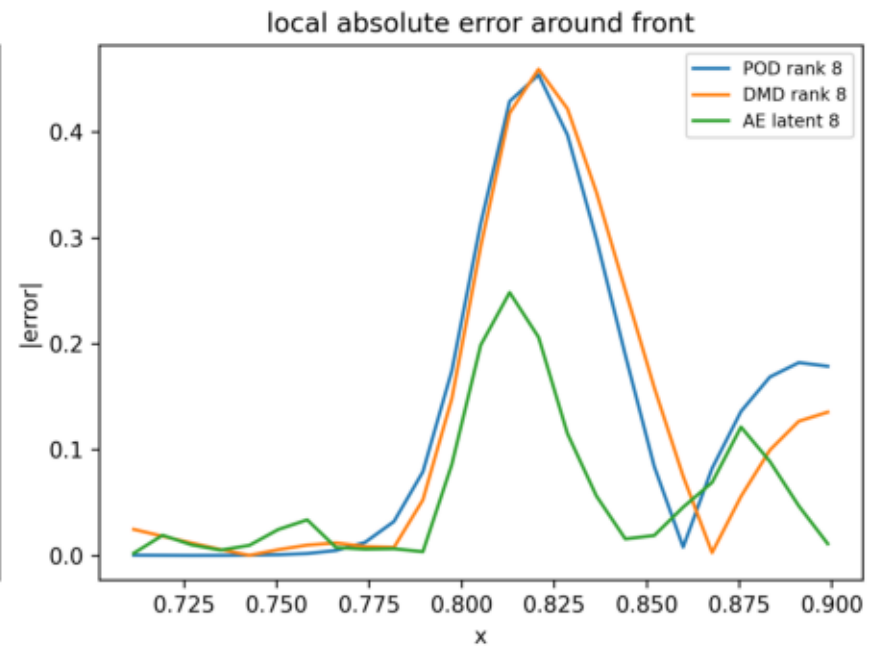
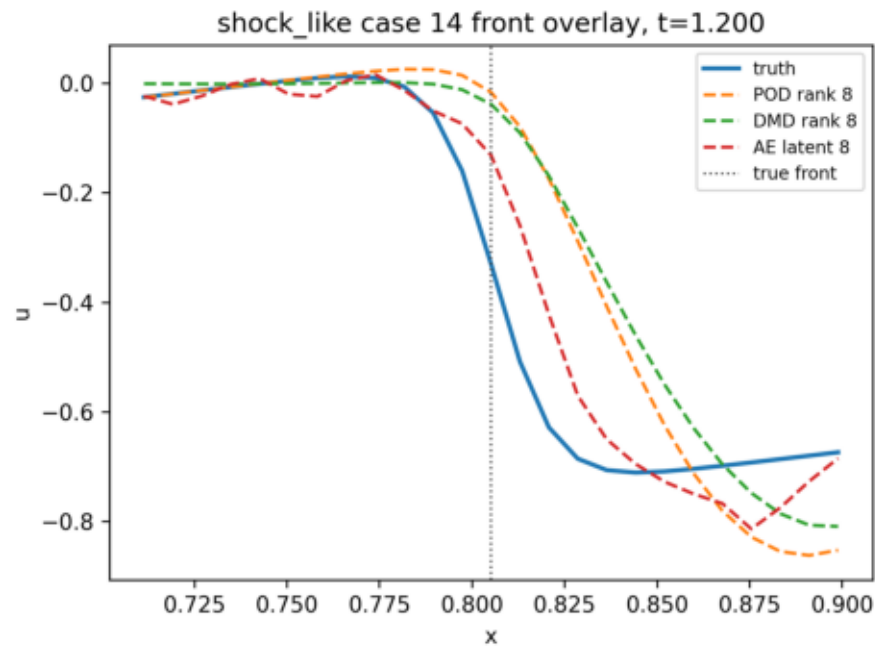
두 case의 true solution을 나란히 보여준다. shock-like case는 공간 기울기가 큰 구조가 훨씬 뚜렷하다.

smooth-like case front 주변 overlay



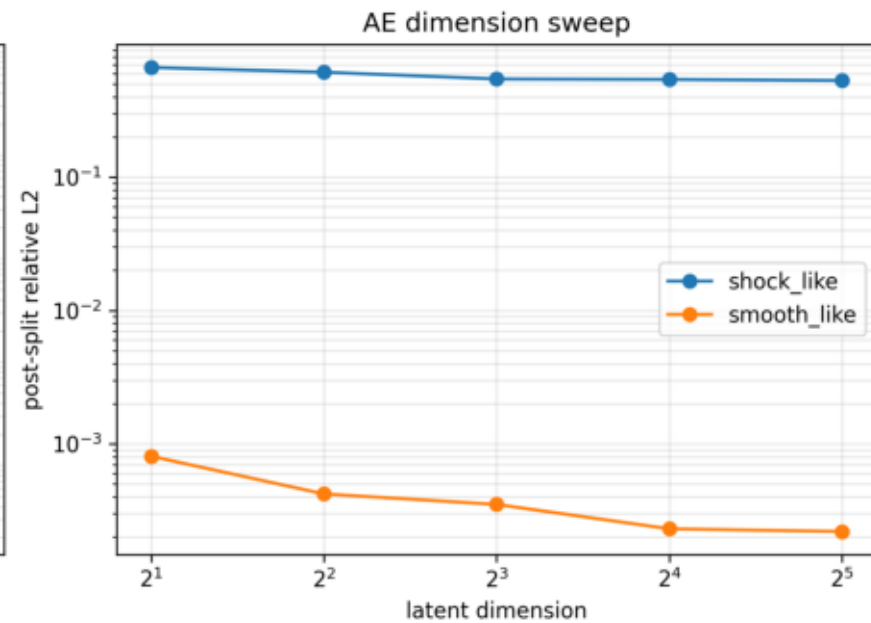
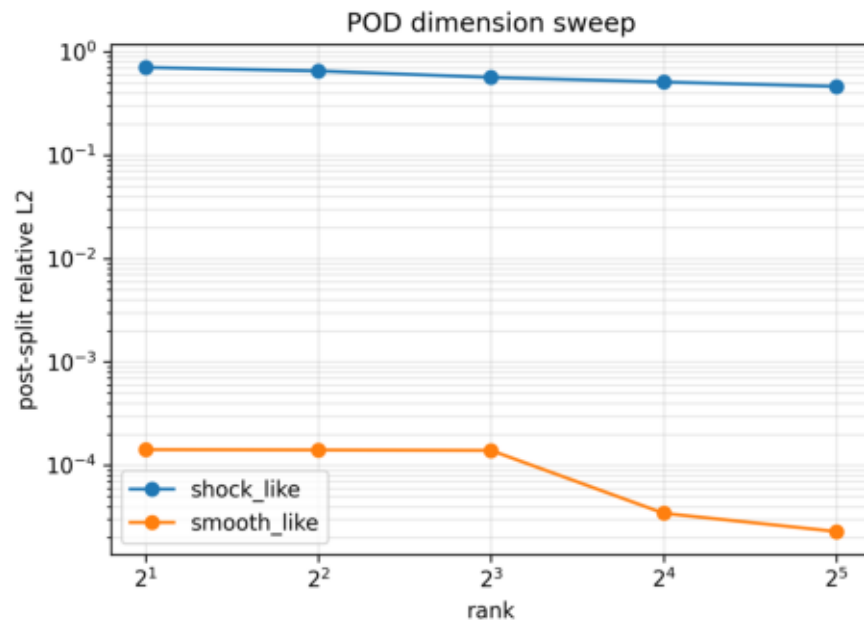
smooth-like case에서 truth, POD rank 8, DMD rank 8, AE latent 8 결과를 front 주변에서 겹쳐 그렸다.

shock-like case front 주변 overlay



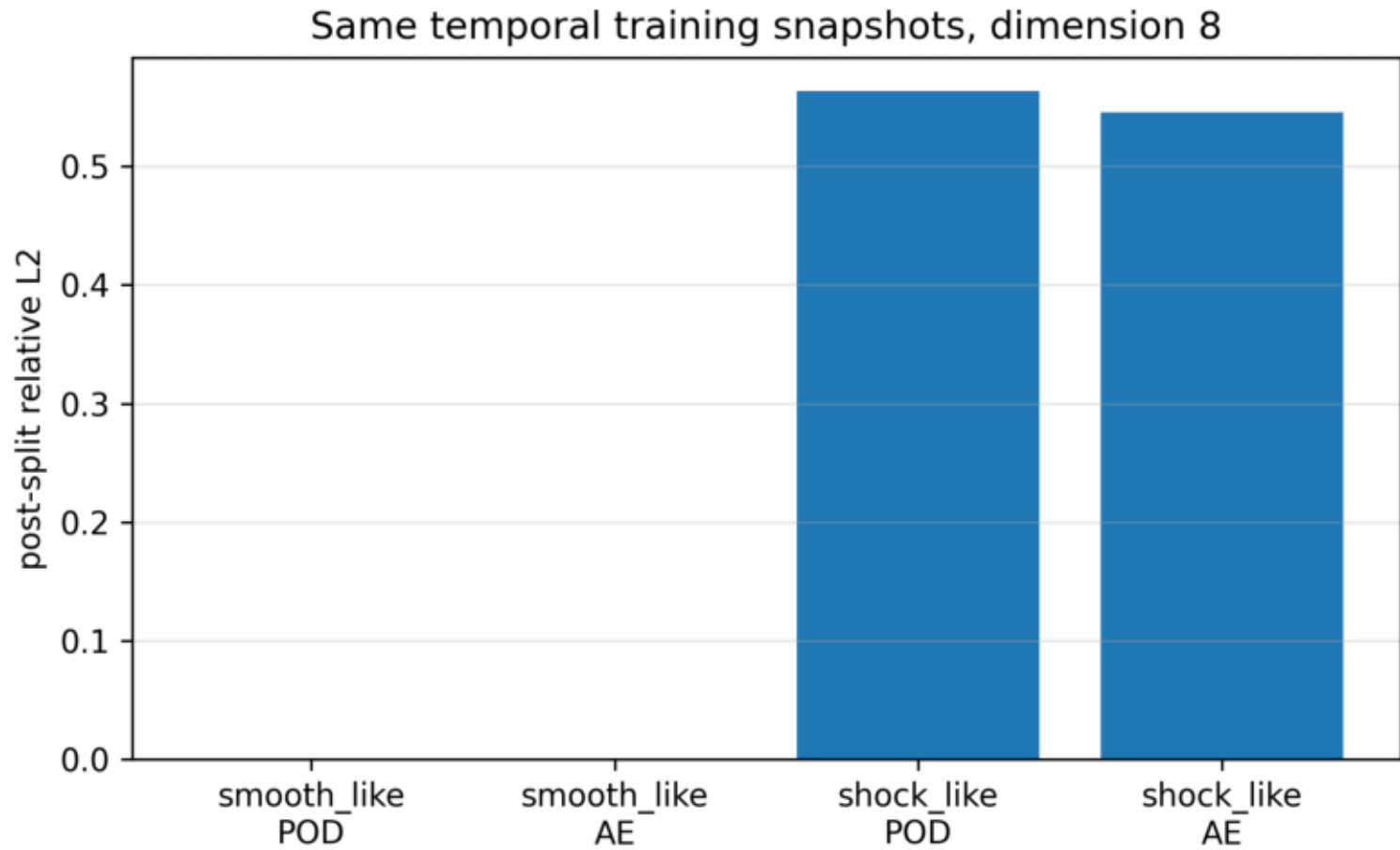
shock-like case에서 truth, POD rank 8, DMD rank 8, AE latent 8 결과를 front 주변에서 겹쳐 그렸다. local absolute error도 함께 표시했다.

POD rank / AE latent dimension sweep



POD rank와 AE latent dimension을 2, 4, 8, 16, 32로 바꿨을 때 post-split relative L2가 어떻게 달라지는지 smooth-like와 shock-like case로 나누어 표시했다.

동일 학습 조건 reconstruction 비교



POD와 AE가 같은 temporal training snapshots를 사용하도록 맞추는 controlled comparison이다. 이 그림도 순위가 아니라 조건을 맞추는 관찰 결과로 읽어야 한다.

19. 판단 결과가 왜 그렇게 나왔는가: POD

POD rank 8의 reconstruction relative L2는 $3.096073e-01$ 이다. 이 값은 train split 전체에서 얻은 8개의 선형 basis가 held-out case 14의 전체 시간장을 어느 정도 표현했는지를 보여준다.

Burgers 해가 단순한 진폭 변화만 한다면 적은 POD mode로도 잘 표현된다. 하지만 front 또는 sharp gradient가 위치를 바꾸면, 같은 모양이 조금 이동한 것만으로도 선형 basis 입장에서는 여러 mode가 필요하다. 그래서 rank 8에서는 front 주변 오차와 gradient 오차가 남는다.

front_position_mae는 $1.066677e-01$, spatial-gradient relative L2는 $8.752090e-01$ 이다. 이는 front 위치 오차와 gradient sharpness 오차가 서로 다른 정보를 준다는 뜻이다.

또 하나 중요한 점은 POD의 final error가 rollout 실패를 의미하지 않는다는 것이다. 현재 POD는 매 시점 true snapshot을 basis에 projection하므로 시간 적분 오차가 쌓이지 않는다. 그래서 POD 결과는 '동일 rank에서 선형 공간이 데이터를 얼마나 담는가'를 보는 결과다.

따라서 이 결과는 'POD 로직은 reconstruction 목적에 맞고, rank 8 선형 subspace에서 moving front 구조가 어떤 오차 양상으로 나타나는지 관찰할 수 있다'로 해석한다.

20. 판단 결과가 왜 그렇게 나왔는가: DMD

DMD reconstruction relative L2는 $1.710189e-01$ 이다. 이것은 DMD가 case 14의 시간 데이터 자체에서 modal dynamics를 맞추기 때문에 학습 구간의 시간 패턴을 어떻게 재구성했는지 보여준다.

하지만 rollout relative L2는 $8.867985e-01$, final rollout error는 $1.258604e+00$ 로 커진다. Burgers

방정식의 실제 시간 발전에는 $u \, du/dx$ 라는 nonlinear 항이 있는데, DMD는 이를 고정된 linear operator 하나로 근사한다.

짧은 시간에는 선형 근사가 그럴듯하게 보일 수 있다. 그러나 시간이 길어지면 front 위치, amplitude, phase, diffusion에 의한 smoothing 차이가 조금씩 쌓인다. 이 누적 오차가 rollout error 증가로 나타난다.

threshold crossing time은 0.0000 근처로 기록되었다. 이 시점 이후에는 field 전체의 shape 차이나 phase 차이가 평가 threshold를 넘어섰다고 볼 수 있다.

front_position_mae가 $3.972942e-01$ 로 기록되었다고 해서 field 전체가 같은 형태로 예측되었다는 뜻은 아니다. front의 대표 위치는 비슷해도, 앞뒤 profile의 amplitude나 shock-like gradient가 틀리면 L2 error는 달라질 수 있다.

따라서 이 결과는 'DMD 로직은 prediction 실험에 맞고, nonlinear Burgers dynamics를 linear rollout으로 근사할 때 어떤 누적 오차가 생기는지 관찰할 수 있다'로 해석한다.

21. 판단 결과가 왜 그렇게 나왔는가: Autoencoder

Conv1D AE의 reconstruction relative L2는 $2.841150e-01$ 이다. 같은 latent_dim=8 조건에서 encoder와 decoder의 nonlinear mapping이 snapshot을 어떻게 표현하는지 보여준다.

POD는 rank 8 선형 평면 안에서 답을 찾는다. 반면 Autoencoder는 8차원 latent vector를 거쳐 decoder가 nonlinear하게 field를 만든다. 그래서 두 방법은 같은 숫자 차원이라도 표현 방식이 다르다.

AE latent linear rollout relative L2는 $8.806610e-01$, final rollout error는

$1.036132e+00$ 이다. 이 값은 AE가 기본적으로 reconstruction loss를 줄이도록 학습되었고, latent dynamics는 별도 선형 근사로 붙어 있음을 보여준다.

reconstruction을 잘하는 latent 좌표가 반드시 시간에 따라 직선적이거나 선형 시스템처럼 움직이는 좌표는 아니다. latent trajectory가 휘어 있거나 서로 얹혀 있으면, linear dynamics가 미래 latent를 잘못 보낼 수 있다.

또한 rollout 중 latent vector가 학습 때 보지 못한 영역으로 벗어나면 decoder는 학습 분포 밖의 field를 만들 수 있다. 이 off-manifold 현상이 final rollout error에 영향을 줄 수 있다.

따라서 이 결과는 'AE reconstruction 로직과 latent linear rollout 로직을 분리해서 보며, nonlinear representation과 latent dynamics의 차이를 관찰한다'로 해석한다.

22. 최종 검토 의견

코드 로직 자체는 현재 목표에 대해 대체로 맞다. 공개 PDEBench 데이터만 사용하도록 정리되었고, 세 방법 모두 같은 processed HDF5와 같은 held-out case를 사용한다.

다만 결과 판단은 반드시 두 층으로 나누어야 한다. 첫째, reconstruction 문제에서는 POD와 AE가 어떤 방식으로 snapshot을 표현하는지 본다. 둘째, prediction 또는 rollout 문제에서는 DMD와 AE latent dynamics의 오차 누적 양상을 본다. POD projection 숫자를 DMD rollout 숫자와 직접 경쟁시키면 공정하지 않다.

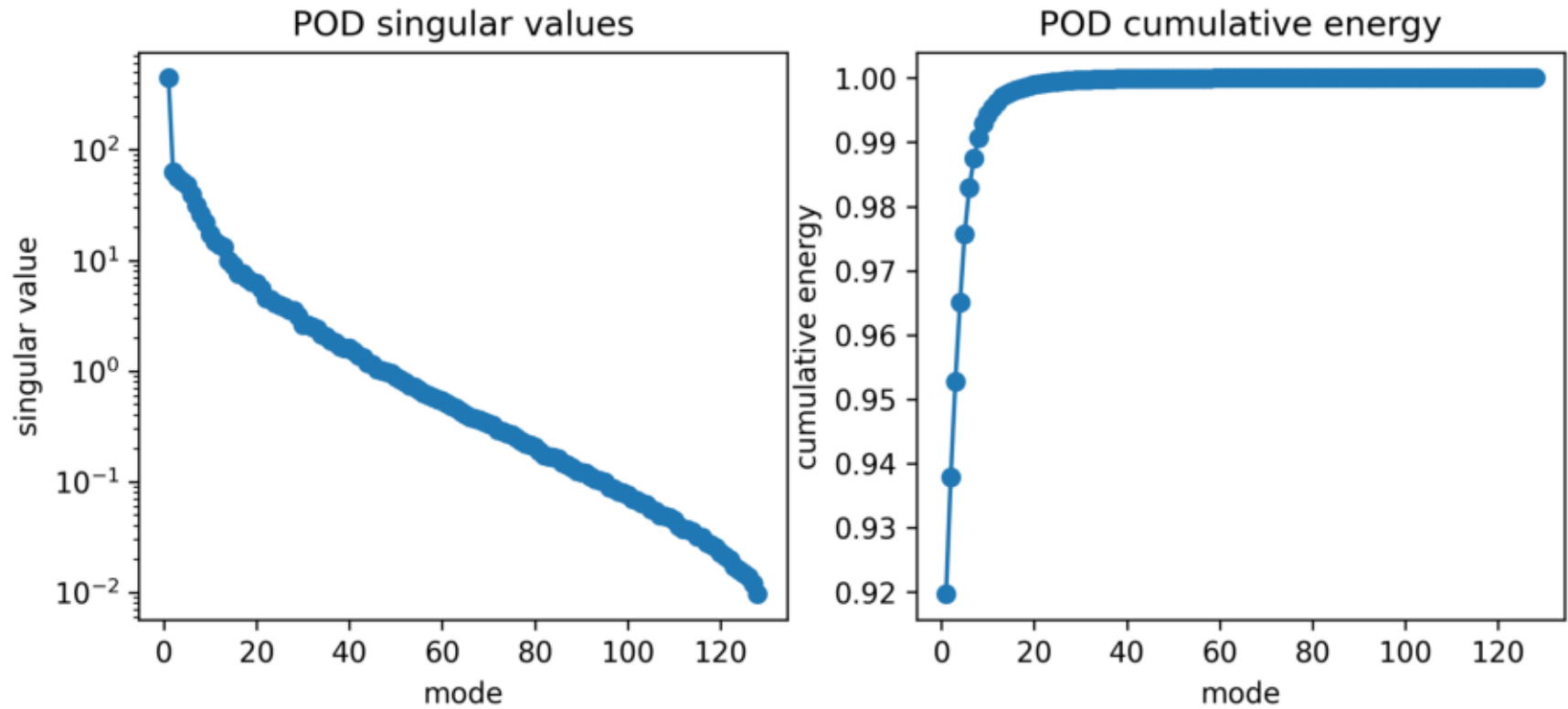
현재 수치는 각 방법의 관찰 지표다. DMD reconstruction 값은 선택된 case의 시간 구조에 직접 맞춰진 modal reconstruction 결과이고, AE reconstruction 값은 nonlinear encoder-decoder의 snapshot 복원 결과이며, POD 값은 train basis projection 결과다.

결론적으로, 이 실험은 방법의 순위를 정하려는 결과가 아니다. 더 정확한 결론은 '같은 공개 데이터에서 reconstruction과 rollout은 서로 다른 문제이며, nonlinear Burgers 구조 때문에 각 방법의 오차 양상이 다르게 드러난다'이다.

POD Figures

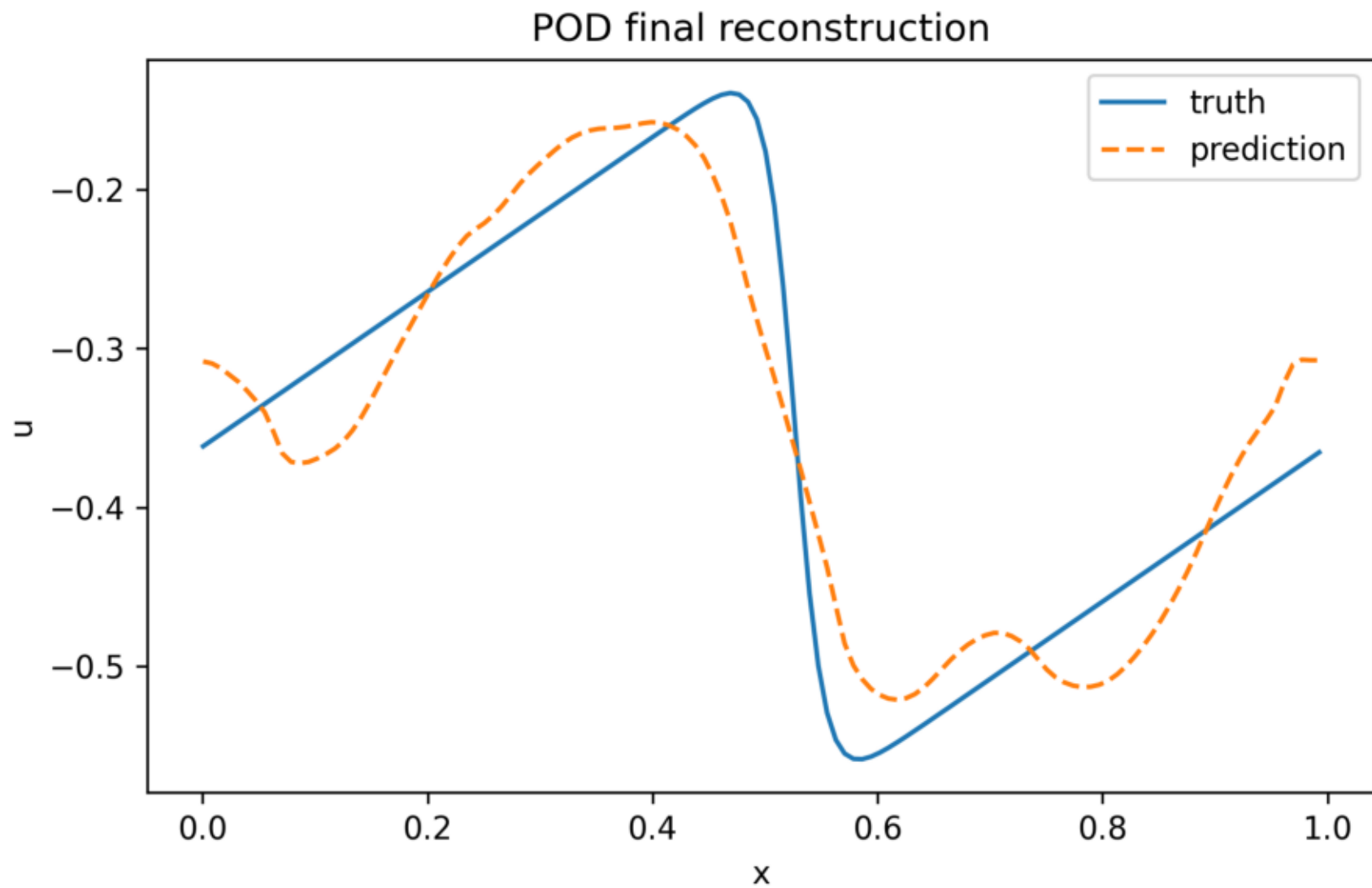
아래 그림들은 public PDEBench subset에 POD 방법을 적용해 생성한 결과다.
각 그림은 원본 PNG 파일을 PDF 뒤쪽에 한 장씩 붙인 것이다.

POD: Pod Energy Spectrum



POD singular value spectrum과 cumulative energy. Rank 선택과 에너지 집중 정도를 확인한다.

POD: Pod Final Reconstruction



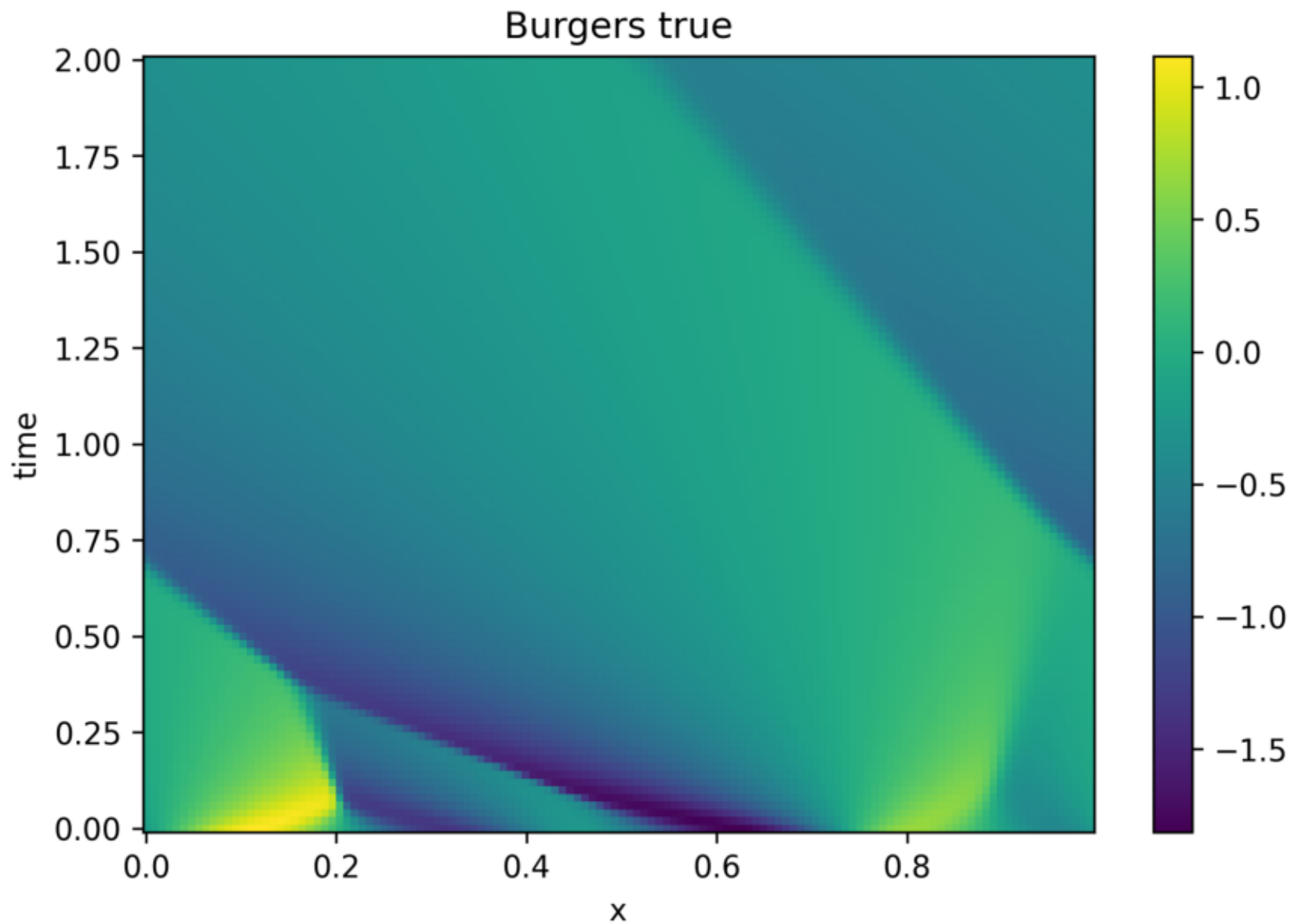
Held-out case 14의 마지막 snapshot에서 truth와 POD reconstruction을 비교한다.

POD: Rollout Error Vs Time

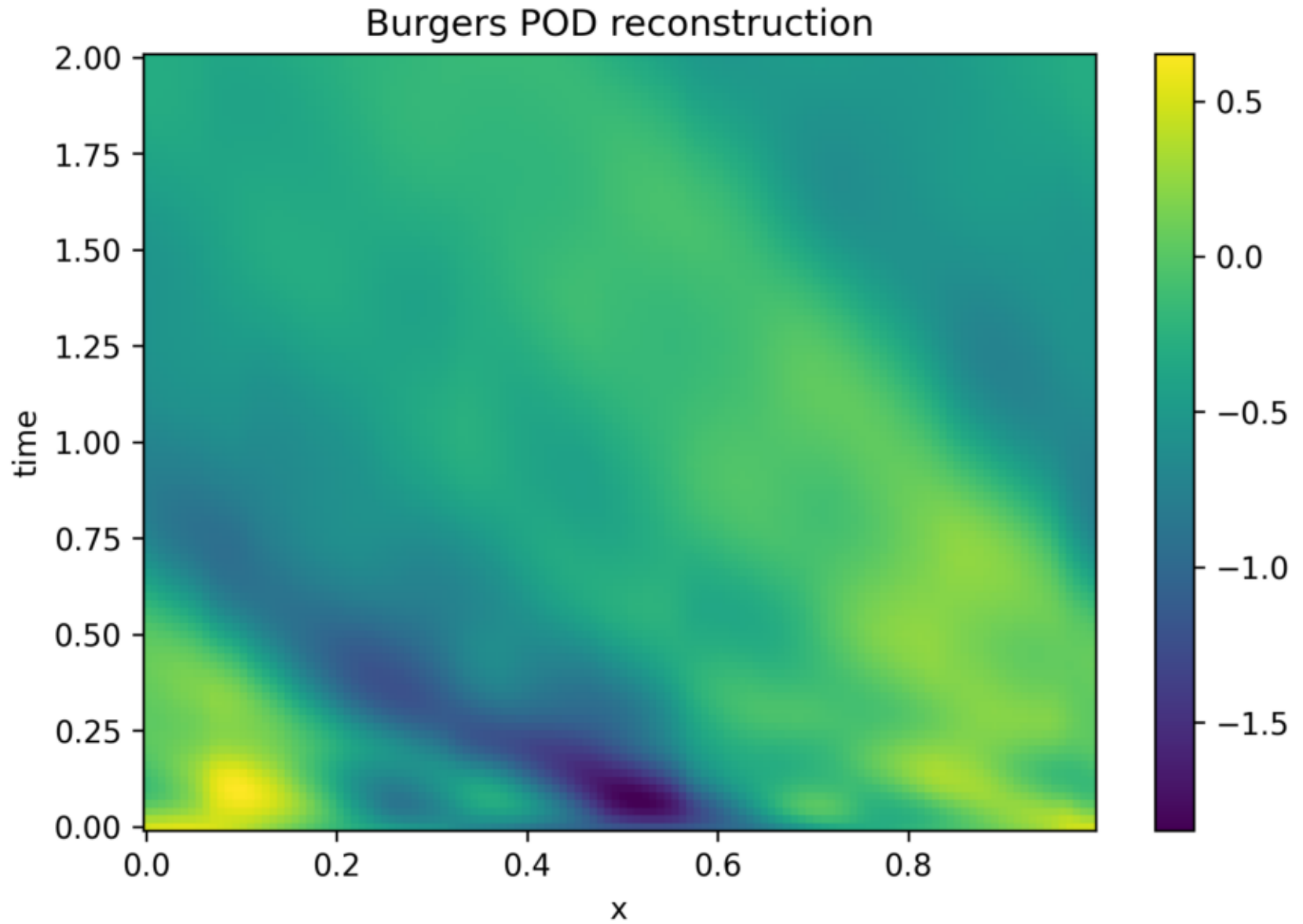


POD projection reconstruction의 시간별 relative L2 error. POD는 rollout이 아니라 snapshot별 projection이다.

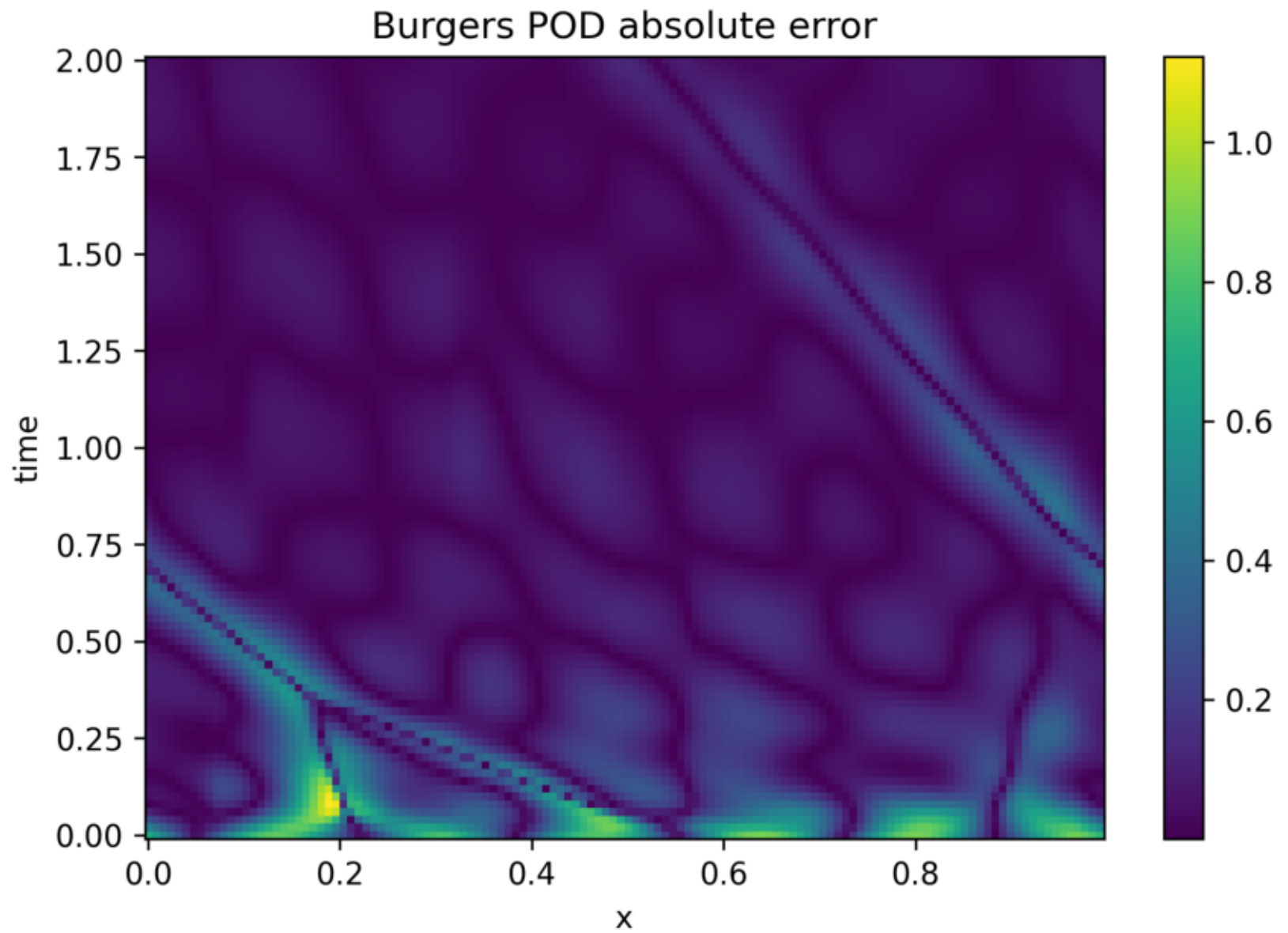
POD: Burgers Spacetime True



POD: Burgers Spacetime Reconstruction



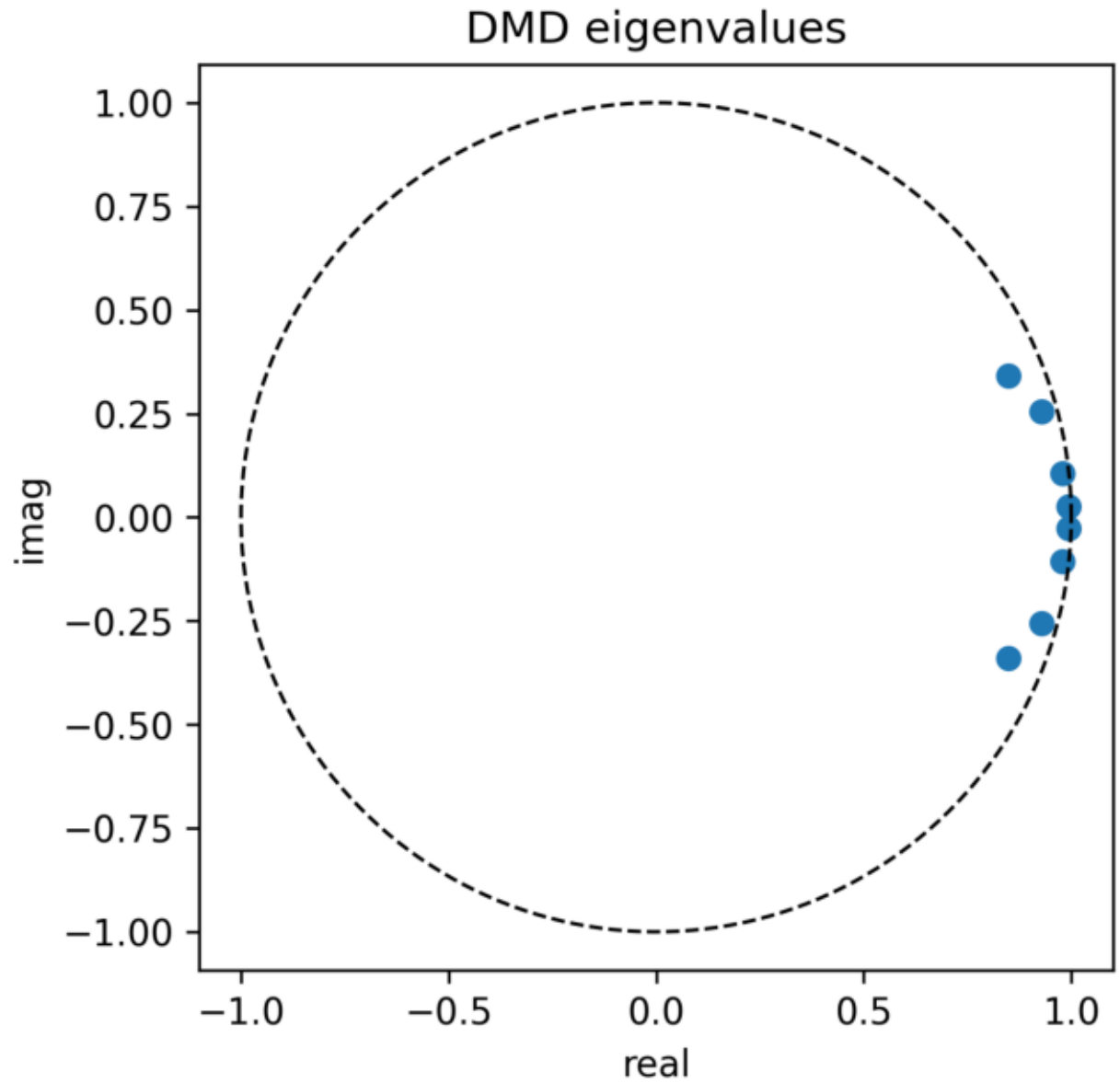
POD: Burgers Spacetime Error



DMD Figures

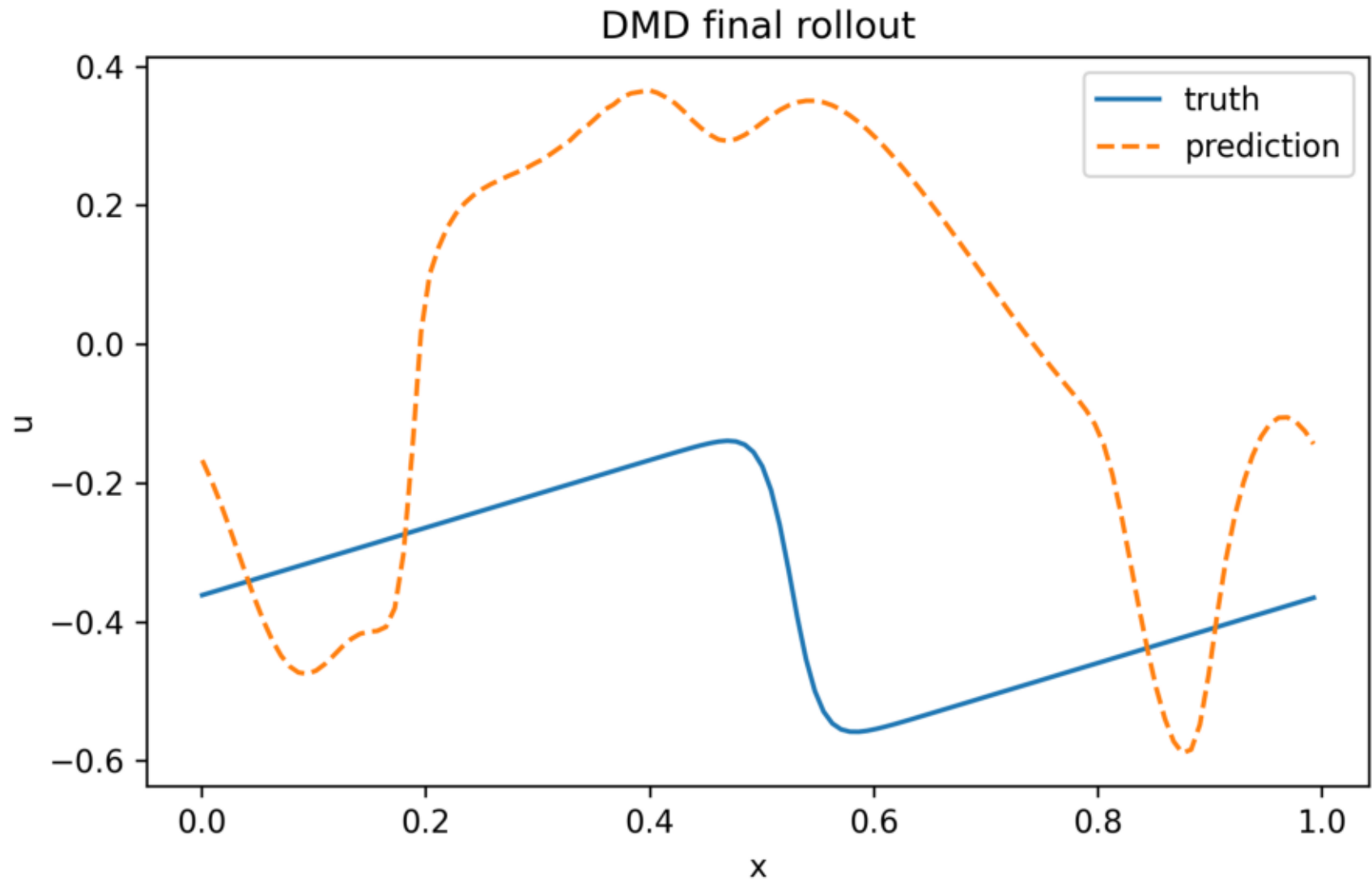
아래 그림들은 public PDEBench subset에 DMD 방법을 적용해 생성한 결과다.
각 그림은 원본 PNG 파일을 PDF 뒤쪽에 한 장씩 붙인 것이다.

DMD: Dmd Eigenvalues



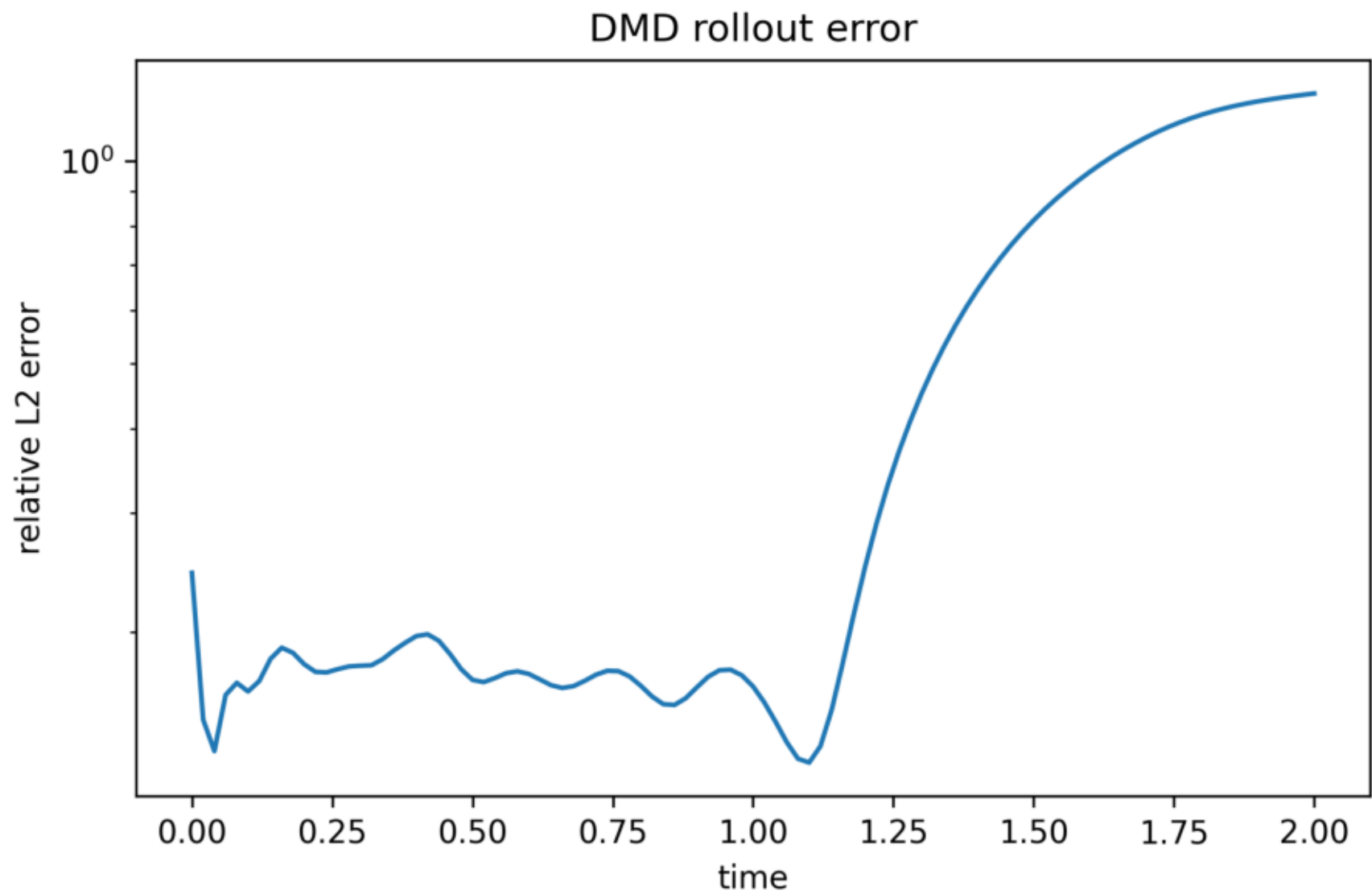
DMD eigenvalue 위치. Rollout 안정성과 mode 감쇠/성장 가능성을 확인한다.

DMD: Dmd Final Rollout

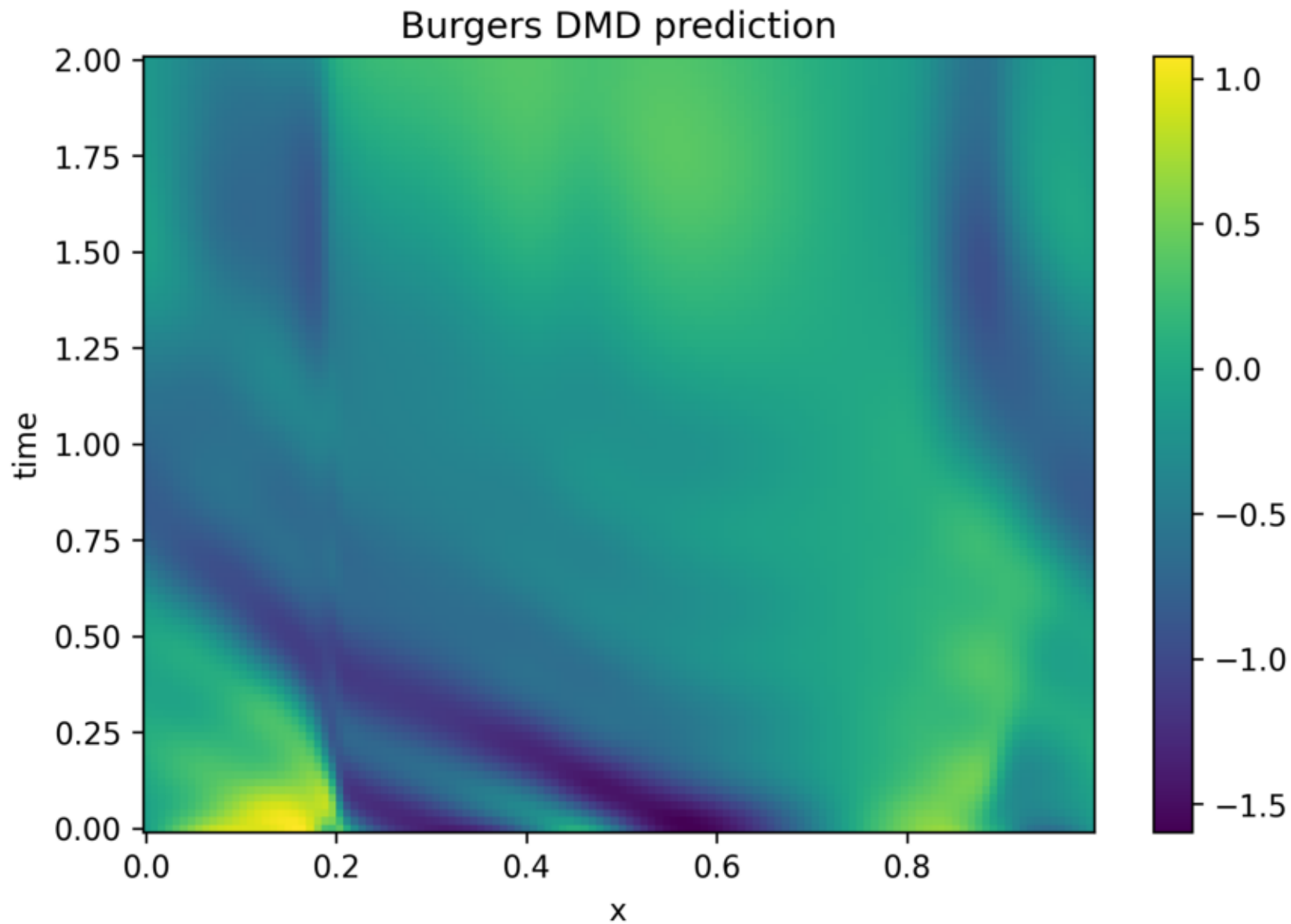


마지막 시점에서 truth와 DMD free rollout prediction을 비교한다.

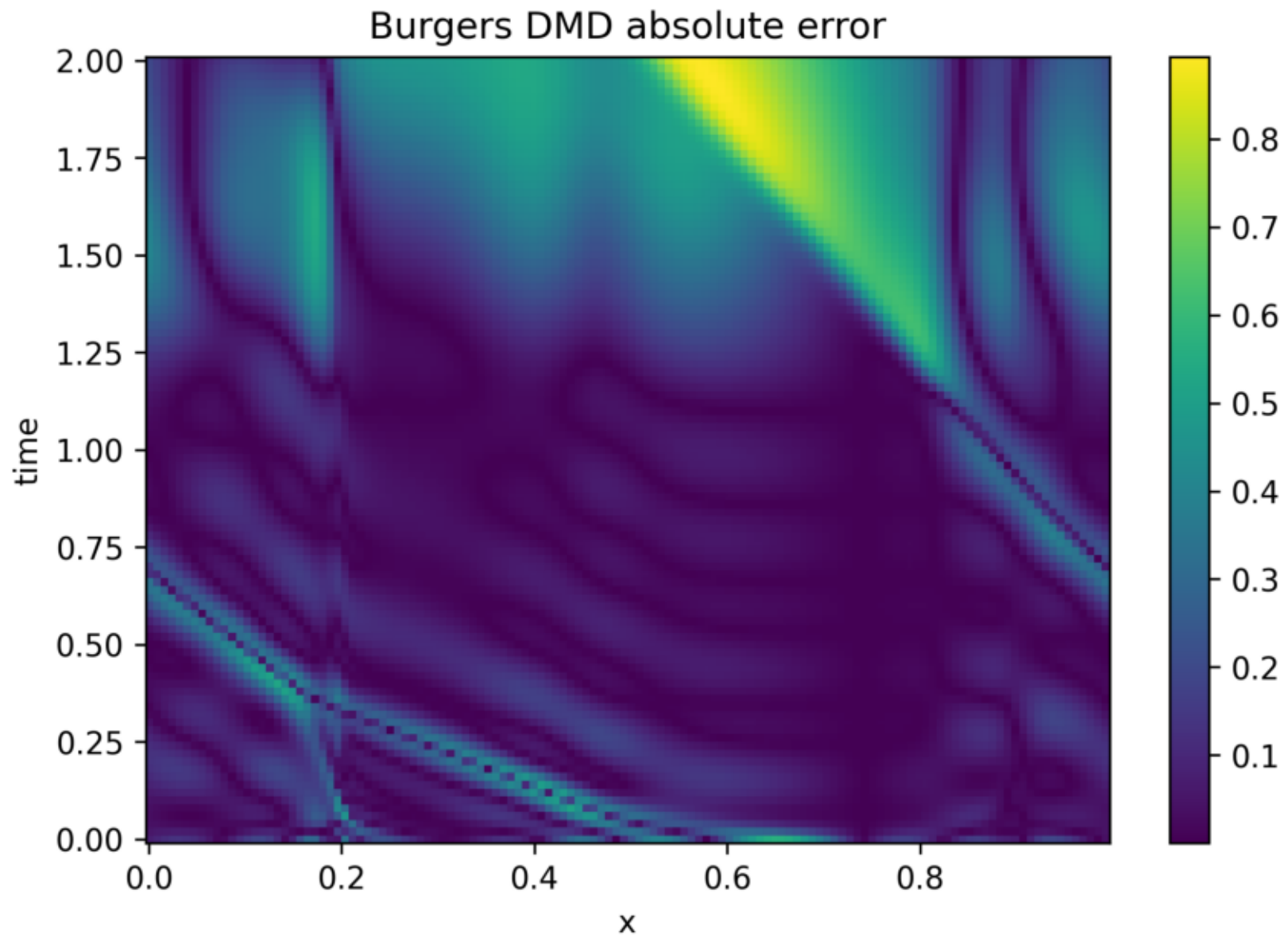
DMD: Rollout Error Vs Time



DMD: Burgers Spacetime Prediction



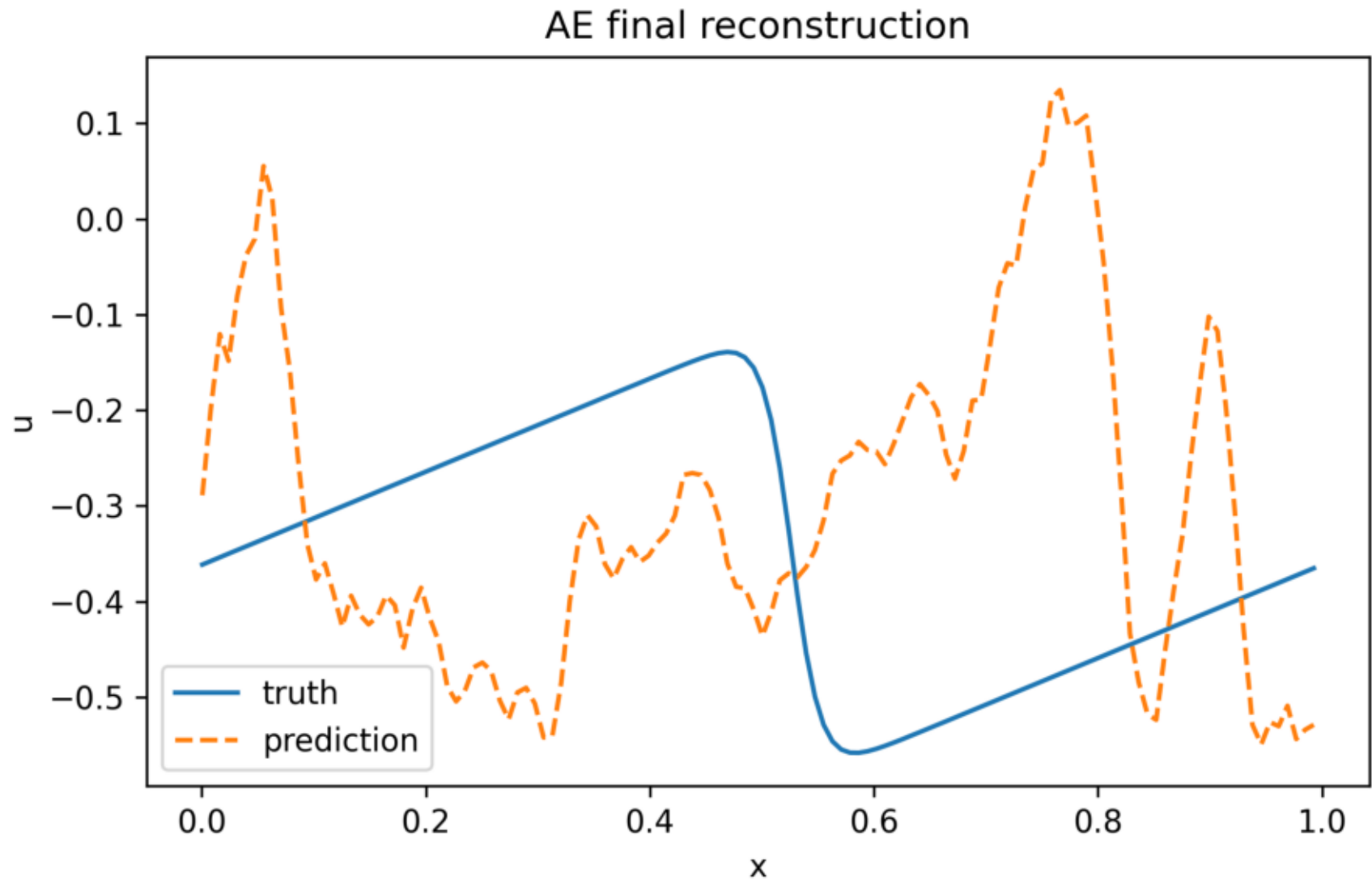
DMD: Burgers Spacetime Error



Autoencoder Figures

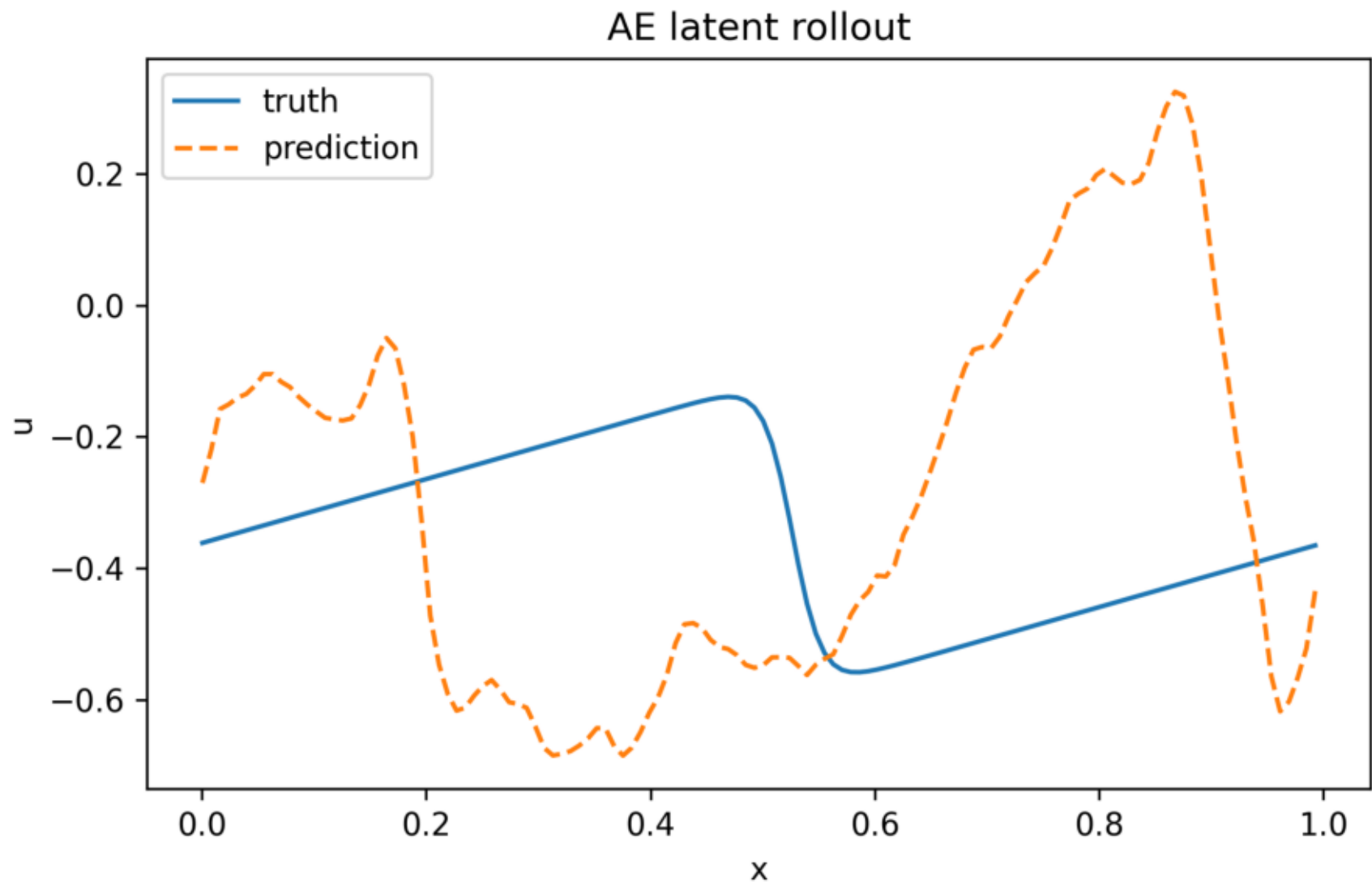
아래 그림들은 public PDEBench subset에 Autoencoder 방법을 적용해 생성한 결과다.
각 그림은 원본 PNG 파일을 PDF 뒤쪽에 한 장씩 붙인 것이다.

Autoencoder: Autoencoder Final Reconstruction



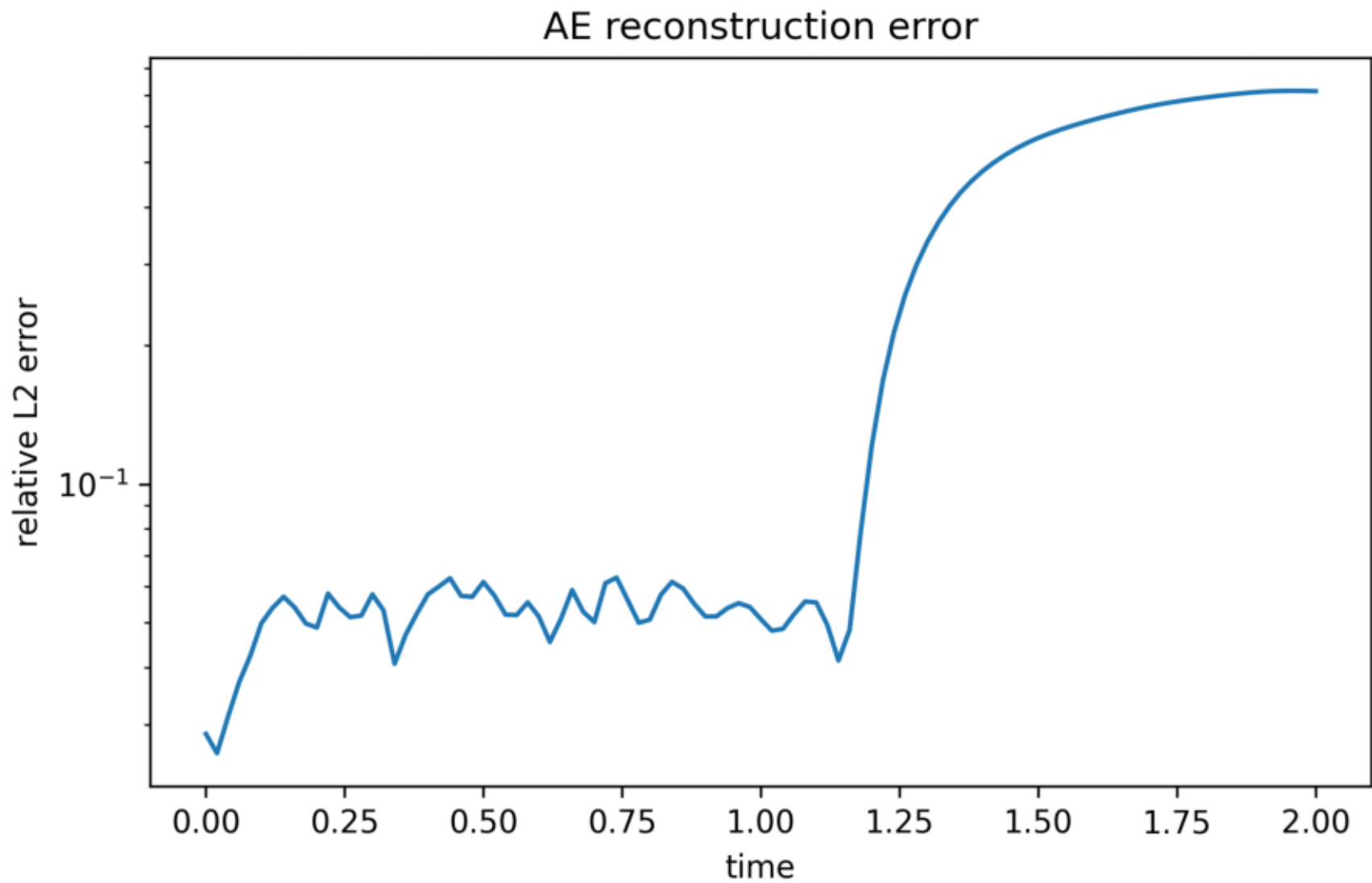
마지막 시점에서 truth와 Conv1D AE reconstruction을 비교한다.

Autoencoder: Autoencoder Latent Rollout Final

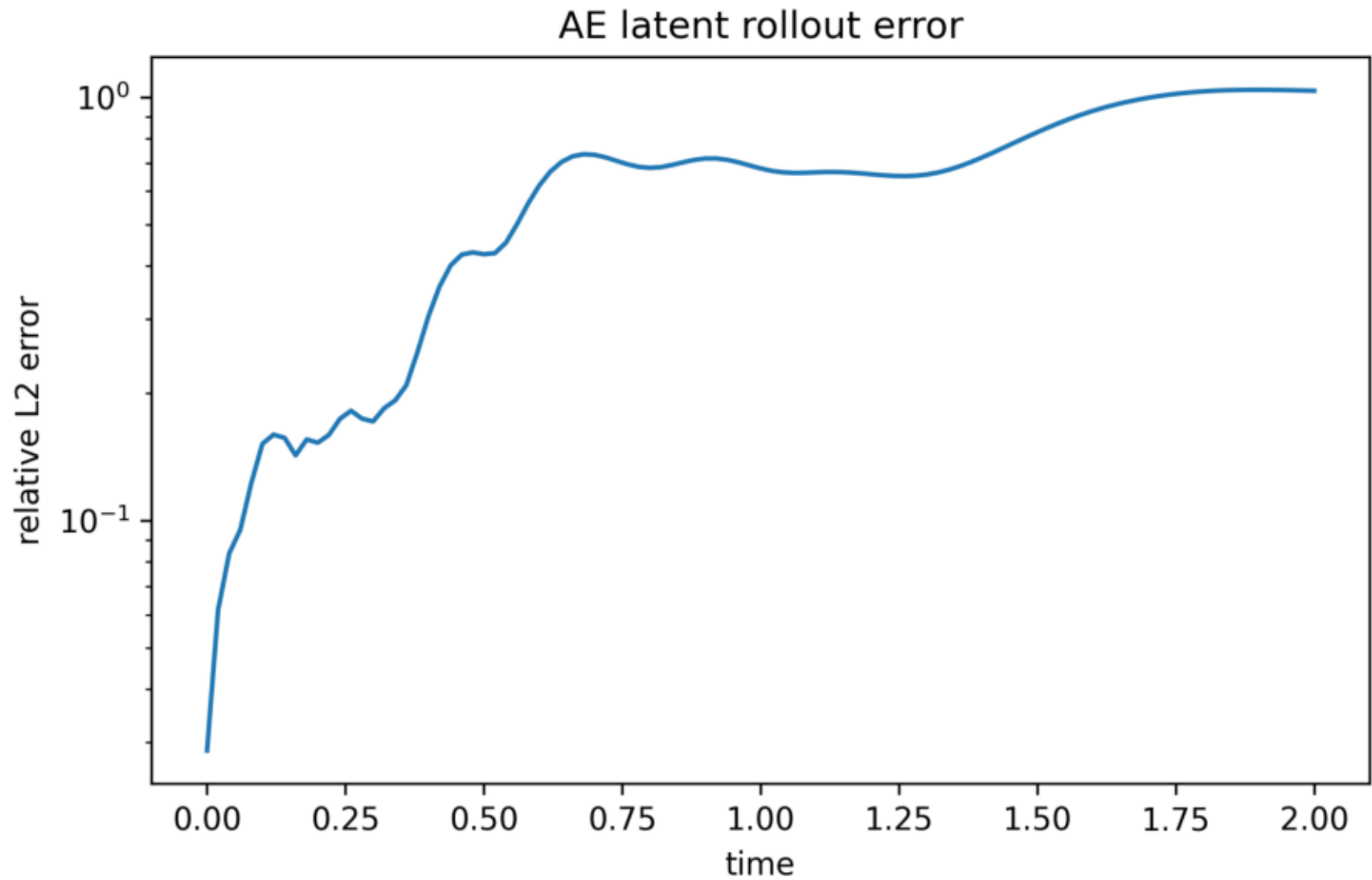


마지막 시점에서 truth와 latent linear rollout 결과를 비교한다.

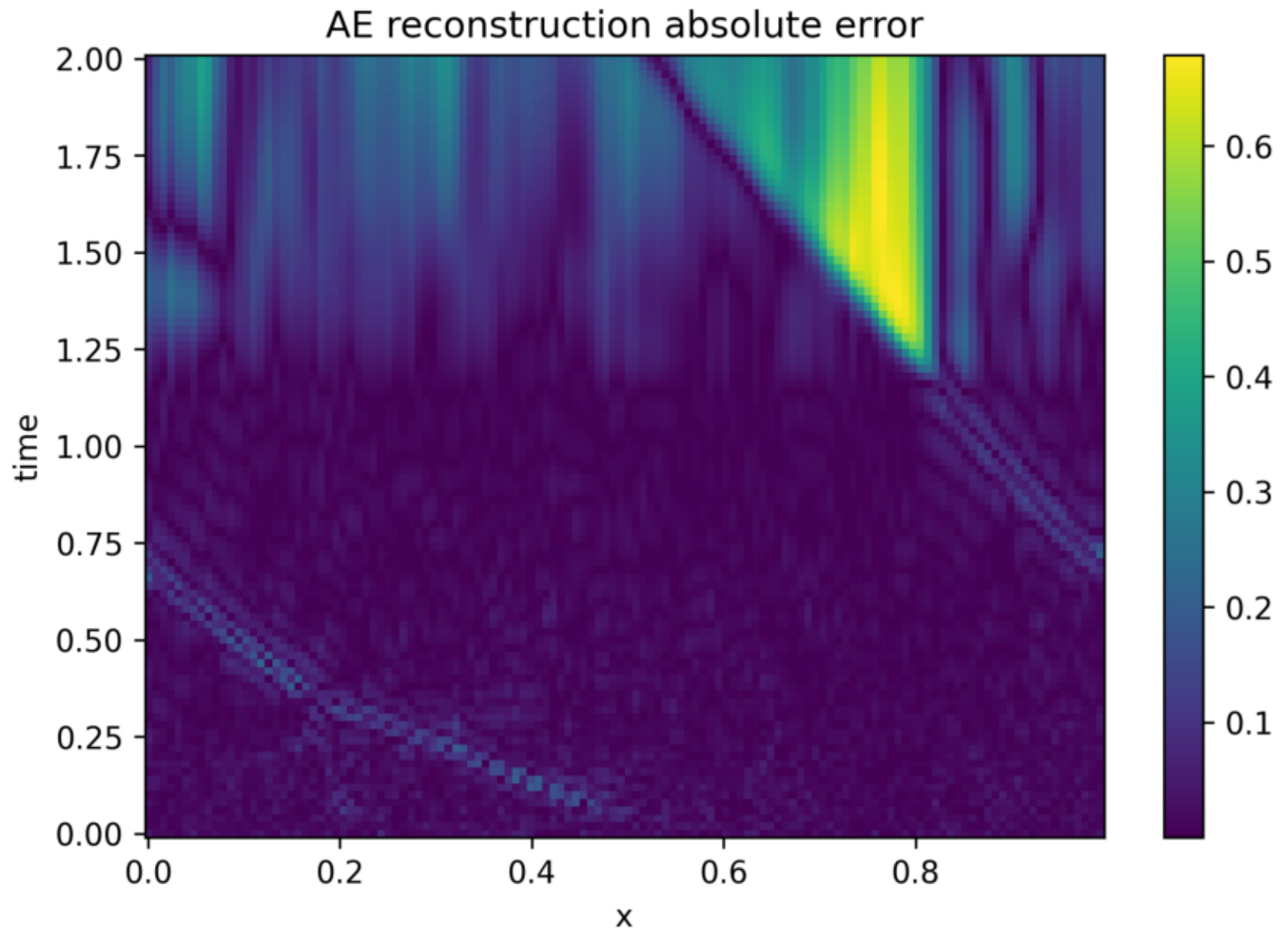
Autoencoder: Reconstruction Error Vs Time



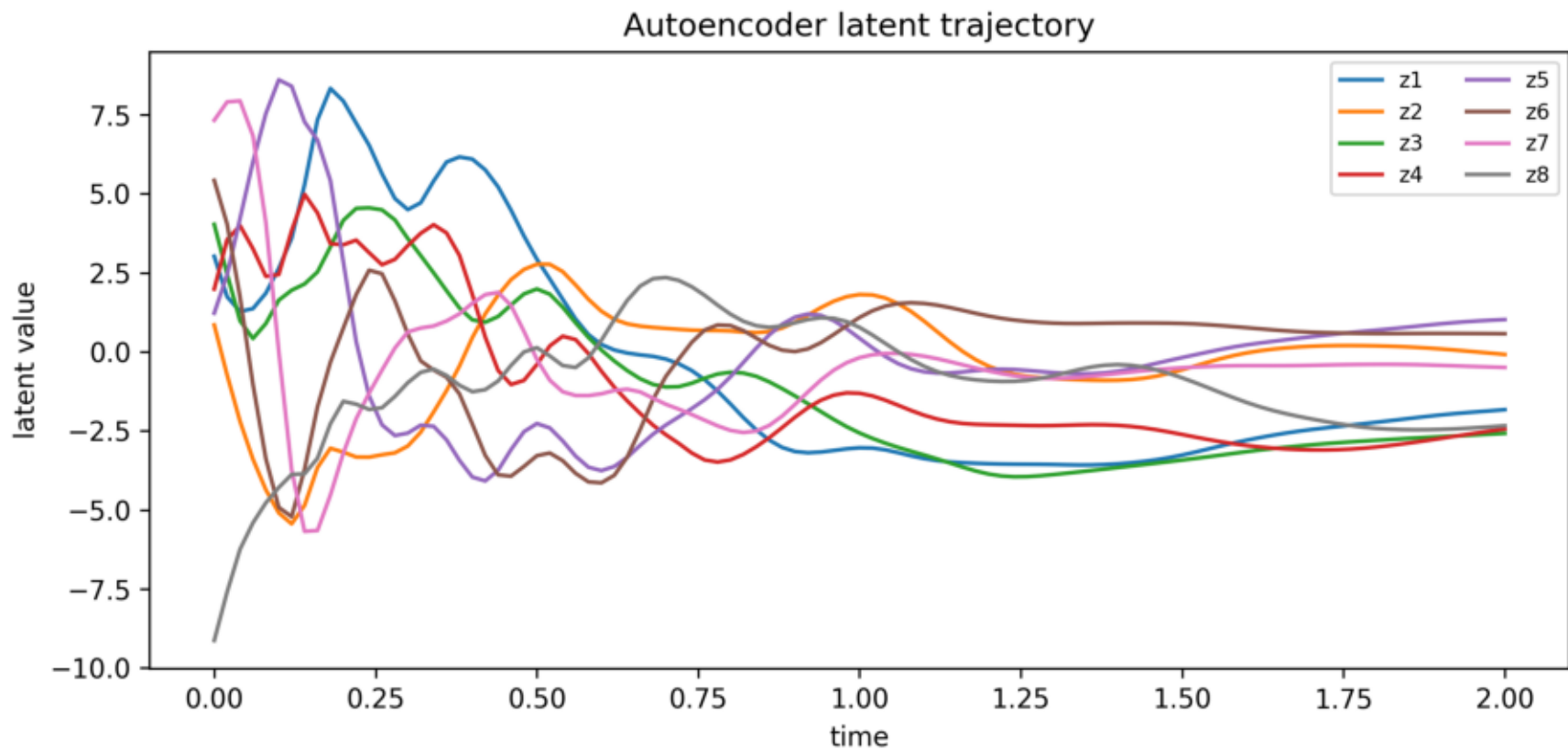
Autoencoder: Rollout Error Vs Time



Autoencoder: Burgers Spacetime Error



Autoencoder: Latent Trajectory



latent_dim=8 좌표의 시간 변화. Latent representation을 정성적으로 확인한다.